Systems Study Books

サービス間通信の設計入門

Service Communication Primer

RPC と Kafka を題材に、サービス間通信をどう設計するかを整理する学習用ノートです。同期通信と非同期通信を対立する流派としてではなく、timeout、retry、ordering、delivery semantics、backpressure、consumer group といった判断軸で並べて理解できるように構成しています。network-io-primer で見た待ち合わせの感覚を、複数サービスが協調する分散システムへ伸ばすのが本書の役割です。

対象読者

HTTP や gRPC の存在は知っているが、RPC、Kafka、consumer 運用、delivery semantics を一本の線で理解したい backend 実務者。

この本で扱うこと

critical path、timeout、retry、idempotency、append-only log、consumer group、lag、DLQ、outbox、replay、playbook。

2026 Edition

この本の使い方

最初から順に読んでもよいですが、実務でぶつかっている痛みから入っても構いません。`request/response` の意味づけに迷っているなら `02` と `03`、Kafka を入れたあとの責務分解で詰まっているなら `04` と `05`、整合性や replay の会話を具体化したいなら `06`、障害対応や ownership を整えたいなら `07` と `08` から読み始めるのが効きます。

本書は製品マニュアルではありません。`gRPC` や Kafka の設定項目を網羅するのではなく、何を同期 `path` に残し、何を非同期へ逃がし、失敗時に誰が責任を持つか を言葉にできる状態を目指します。各章の演習と付録の worksheet は、そのまま設計レビューや runbook づくりに持ち込めるようにしてあります。

> **編集注記**
> この文書は `gRPC` や Kafka の feature list を網羅する製品マニュアルではありません。サービス間通信で本当に難しくなる点、たとえば timeout、partial failure、retry、重複、順序、再処理、監視を、`RPC` と `Kafka` を対比しながら学べるように再構成した版です。設定項目や vendor 差分の完全網羅よりも、「なぜその設計判断が必要か」を説明できる状態を目指します。

目次

Contents

はじめに 10
- この教材の主題 11
- local call の直感はどこで壊れるのか 12
- なぜ RPC と Kafka を一緒に扱うのか 12
- 本書が扱う問い 12
- 対象読者と前提 12
- この本の見方 13
- 本書を通して繰り返す 4 つの問い 13
- 継続例で追う状態境界 14
- accepted、rejected、unknown を分けて読む 14
- 各章の見取り図 14
- 最初に押さえる読み方 15
- この本を読み終えたときの出口 15

request/response と critical path 16
- critical path を先に決める 16
- 継続例: 注文作成の同期部分 16
- response が意味するものを固定する 17
- contract は 3 層ある 18
  - data contract 18
  - temporal contract 18
  - failure contract 18
- timeout と deadline を予算として扱う 18
- 4 つの通しケース 19
  - ケース 1: 正常系 19
  - ケース 2: 在庫切れ 20
  - ケース 3: 支払い timeout 20
  - ケース 4: response loss 後の client retry 20
  - ケース 5: caller は諦めたが backend は遅れて成功した 20
- stable request identity をどこで作るか 20
- status inquiry を設計に含める 21
- retry してよい call と、してはいけない call 21
- idempotency key は誰のためにあるのか 22
- user-facing state は 3 種類に寄せる 22
- user に見せる失敗を設計する 22
- 支払いと在庫の順序をどうするか 23
- response を返したあとに何が残るか 23
- API 名と状態名をずらさない 24
- read-your-own-writes をどこまで求めるか 24
- failure mapping の小さな表 24
- caller cancellation を無視しない 25
- 小さな通しシナリオ: 二重押しからの回復 25
- critical path から外す基準 25
- 章末まとめ 25

この章の設計判断の要点 26
演習 26
この章の出口 26
RPC の運用設計 26
論理名で呼ぶということ 27
load balancing は throughput だけの話ではない 27
どの balancing policy を選ぶか 27
connection pool は invisible queue でもある 28
timeout budget と retry budget 28
circuit breaker は何を守るのか 29
rate limit と concurrency limit 29
unary と streaming の使い分け 29
streaming は queue の代わりではない 30
tracing と metrics は一緒に設計する 30
監視設計を先に持つ 30
deadline を hop ごとに伝える 30
tail latency と hedged request 31
service mesh が解決することと、しないこと 31
deploy 時の draining を軽視しない 31
cache と request coalescing 32
rollout と互換性 32
call graph を短く保つ 32
典型的な設定ミス 32
ケーススタディ 1: payment timeout が急増した日 33
ケーススタディ 2: inventory p99 は悪いが service 自体は healthy 33
dependency ごとに予算表を持つ 33
読み取り系と書き込み系を分ける 34
fallback は成功ではない 34
checkout path の運用レビュー質問 34
章末まとめ 35
この章の設計判断の要点 35
演習 35
この章の出口 35
Kafka、log、producer 設計 35
どの仕事を同期 path から外すのか 36
command と event を区別する 36
queue ではなく log として理解する 36
topic、partition、offset 37
partition key は business decision である 37
event schema は何を残すべきか 38
producer が決めるべきこと 38
outbox row に何を持つか 38
append 成功と処理完了は違う 39
producer retry と重複 39
ケース 1: order row はあるが event がまだ見えない 39
event time と processing time 40

topic をどう切るか 40
thin event が同期依存を呼び戻す 40
schema evolution を先に考える 41
retention と replay cost を見積もる 41
compaction が向く場面、向かない場面 41
producer 側にも backpressure がある 42
relay は producer の一部である 42
Kafka は非同期 fan-out の土台 42
producer 設計の比較表 42
topic 命名と ownership を先に決める 43
producer 設計レビューの問い 43
章末で見る producer の責務 43
章末まとめ 43
この章の設計判断の要点 44
演習 44
この章の出口 44
consumer、lag、backpressure 44
3 つの consumer は同じではない 44
consumer group は何を分担するのか 45
offset commit の意味 45
consumer の並列化はどこで行うのか 46
lag を設計信号として読む 46
rebalancing のコスト 47
poison message は「壊れた 1 件」では済まない 47
DLQ と redrive 47
idempotent consumer の設計 48
ケース 1: Email Worker で二重送信が起きた 48
ケース 2: Analytics lag と hot partition 49
ケース 3: Search Indexer の作り直し 49
backpressure はどこでかけるか 49
consumer observability 49
retry topic と delay queue 50
stuck partition を見分ける 50
commit batch と重複窓 50
handler 内の下流 RPC をどう守るか 50
pause/resume を運用に組み込む 51
ケース 4: replay と通常 traffic がぶつかる 51
ケース 5: Email provider が rate limit した日 51
consumer ごとの redrive 方針を分ける 51
lag を見たときの 5 分トリアージ 52
章末で見る consumer の責務 52
batch handler と partition-local state 52
redrive の実務手順 52
consumer 設計レビューの問い 53
章末まとめ 53
この章の設計判断の要点 53

演習 53
この章の出口 54
delivery semantics、ordering、整合性 54
semantics は transport のラベルではない 54
at-most-once の具体像 54
at-least-once の具体像 55
exactly-once をどう読むか 55
ordering scope を決める 55
reorder は自然に起きる 56
dedup のやり方 56
idempotency key の寿命を決める 56
end-to-end invariant を先に書く 57
outbox pattern 57
outbox が解決しないこと 57
saga と補償 58
具体例: 注文の end-to-end timeline 58
ケース: accepted だがメールはまだ送られていない 58
replay と backfill 59
read-your-own-writes と projection 59
freshness budget を product requirement にする 59
cancellation event をどう扱うか 59
inquiry path を consistency の一部として見る 60
projection rebuild と snapshot 60
saga state をどこに持つか 60
manual recovery path を先に決める 60
unknown state を短く保つ 61
状態機械として見る 61
補償が効きにくい操作 61
consistency budget を言葉にする 61
consistency レビューの問い 62
章末まとめ 62
consistency を product と共有する 62
この章の設計判断の要点 62
演習 62
この章の出口 63
障害対応と設計 playbook 63
partial failure を通常系として扱う 63
cascading failure を防ぐ原則 63
インシデント 1: payment timeout が急増した 64
インシデント 2: analytics lag だけが増えている 64
インシデント 3: schema 変更で一部 consumer が落ちた 64
最初の 15 分でやること 65
alert を症状と原因で分ける 65
RPC と Kafka の使い分け表 65
観測を hop ごとと end-to-end で分ける 66
ownership と境界 66

反パターン 66
設計レビューの観点 67
段階的に改善するには 67
runbook に残すべきこと 67
redrive と rollback を混同しない 68
ケーススタディ: 同期 fan-out から event 化へ 68
移行のロールアウト順序 68
degraded mode の棚卸し 69
architecture review worksheet 69
playbook をチームで運用する 69
障害演習の観点 70
postmortem で残すべきもの 70
30 分インシデントタイムライン 70
on-call の役割分担を決める 71
consumer lag 用の runbook 例 71
schema rollout 前の確認 71
game day で試す題材 72
playbook の成熟度を測る 72
章末まとめ 72
この章の設計判断の要点 72
演習 72
この章の出口 73
付録: 復習、用語、次の一歩 73
章ごとの復習ポイント 73
自己点検用の問い 73
設計レビュー用の簡易チェックリスト 74
クイック比較表 74
本書での表記ルール 75
用語小事典 75
consumer タイプの比較 76
よくある誤解 76
RPC は関数呼び出しの延長にすぎない 76
Kafka は queue の高機能版である 76
exactly-once を使えば重複はもう考えなくてよい 76
DLQ に送れば安全である 76
lag は worker を増やせば解決する 76
ミニケーススタディ 77
ケース 1: 注文確認メールが二重送信された 77
ケース 2: 検索結果への反映が遅い 77
ケース 3: 注文受付は成功するが analytics が欠損する 77
設計レビュー演習 77
30 分レビューの進め方 77
20 の確認質問 78
手元の system でやる小さなワーク 78
ミニケーススタディを増やす 79
ケース 4: DLQ が増えているが user 影響はまだ見えない 79

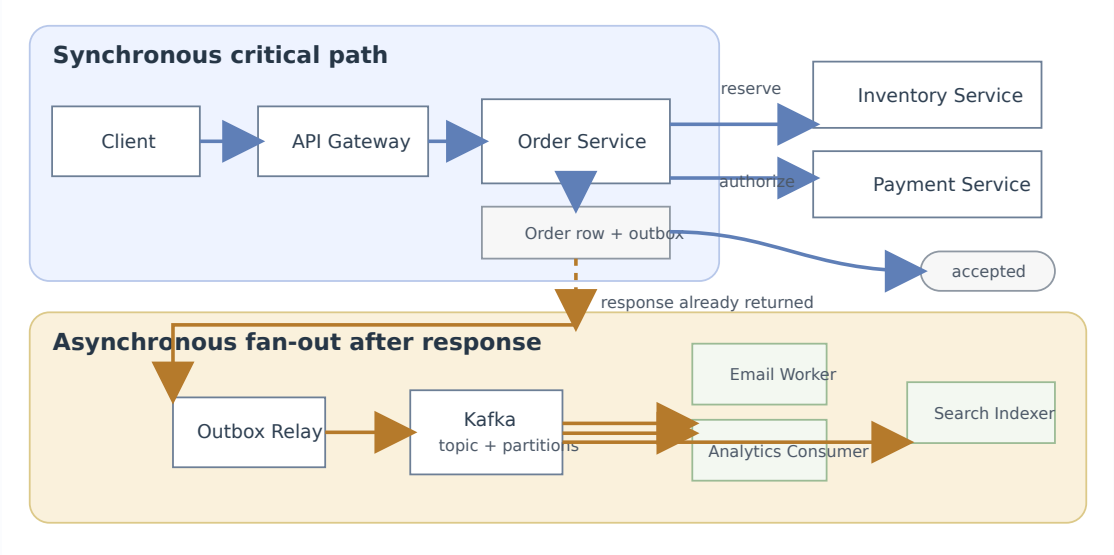
ケース 5: payment provider が断続的に 502 を返す 79
ケース 6: replay 中だけ analytics warehouse が飽和する 79
設計の危険信号 79
設計メモの最小テンプレート 79
runbook の最小テンプレート 80
90 分ワークショップ案 80
学習を深める順序 81
章別の読み返し順 81
参考資料 81
手元で使う 1 ページ worksheet 81
4 週間の使い方 82
おわりに 82

はじめに

この文書は、複数サービスが通信するときに何が難しくなるのかを整理するためのノートです。単一 process の関数呼び出しでは、call して戻れば話が終わります。しかしサービス間通信では、相手は別 process であり、別 host かもしれず、別 deployment cadence と failure mode を持っています。戻り値だけでなく timeout、retry、重複、順序、遅延、再配信、観測可能性まで設計に入ってきます。本書は、その増えた論点を `RPC` と `Kafka` の二つの題材で一本の線にすることを目指します。

`network-io-primer` では、blocking / nonblocking I/O、`select/poll/epoll`、Go runtime の `netpoll` を通して、「待つとは何か」を low-level に整理しました。本書はその次の層です。今度は socket 1 本ではなく、service A と service B、API handler と worker、同期 request と非同期 event のあいだで、どのような約束と失敗が発生するかを見ます。

本書では全章を通して 1 つの継続例を使います。題材は EC サイトの注文システムです。注文 API は `API Gateway` から `Order Service` に入り、`Inventory Service` と `Payment Service` に同期 `RPC` します。そのあと注文確定という事実を `Kafka` に書き込み、`Email Worker`、`Analytics Consumer`、`Search Indexer` がそれぞれ非同期に処理します。つまり、1 つの user 操作の中に、同期通信と非同期通信の両方が入っています。これがサービス間通信の設計を学ぶ題材として都合がよい理由です。



注文システム全体と、同期の critical path、response 後の非同期 fan-out

この構成は特別に複雑なものではありません。むしろ多くの backend system にかなり近い形です。だからこそ学びやすいのです。user にすぐ返したい結果、後でよい副作用、順序を守りたい更新、重複を許容したい処理が、1 つの業務フローの中に同時に出てきます。

この教材の主題

本書の主線は次の 5 点です。

1. `RPC` を「remote function call」ではなく、timeout と失敗を含む request/response として理解する
2. broker と Kafka を、queue ではなく「複数 consumer が読む append-only log」として理解する
3. retry、重複、順序、再処理を、delivery semantics の問題として整理する
4. consumer group、lag、rebalancing、backpressure を、運用の都合ではなく設計の一部として理解する
5. 同期通信と非同期通信を対立させず、system 全体で役割分担させる感覚を持つ

この順序を取る理由は、いきなり Kafka の内部や `gRPC` の API へ飛ぶと、「何を守るためにその仕組みがあるのか」が曖昧になりやすいからです。まず request/response の素朴な世界を見て、そこから broker と log が必要になる理由へ進みます。そのうえで delivery semantics と運用の問題へ戻ると、同期と非同期を比較しやすくなります。

この章の到達点

本書を読み終えるころには、少なくとも次を説明できる状態を目指します。

- `RPC` と local call がどこで決定的に違うか
- timeout、retry、deadline、idempotency をどう組み合わせるべきか
- Kafka の `topic`、`partition`、`offset`、consumer group が何を表すか
- at-most-once / at-least-once / exactly-once を end-to-end でどう解釈するか
- `RPC` と Kafka を同じ system の中でどう使い分けるか

local call の直感はどこで壊れるのか

分散システムの最初のつまずきは、local call の直感をそのまま持ち込むことです。関数呼び出しなら、「呼んだ」「戻ってきた」「失敗した」がかなり明確です。しかしサービス間通信では、そのあいだに DNS、load balancer、sidecar、connection pool、thread pool、queue、broker、worker が入ります。途中のどこかが遅くても caller からはただ timeout に見えるかもしれませんし、callee は成功したのに response だけ失われるかもしれません。

この違いを雑に扱うと、設計はすぐに壊れます。retry は成功率を上げるどころか重複実行を増やし、Kafka は疎結合どころか状態遷移を曖昧にし、監視は green なのに user 体験は壊れる、といったことが起きます。本書は、そこを技術名ではなく設計判断の問題として整理するための本です。

なぜ RPC と Kafka を一緒に扱うのか

分散システムの本は、RPC 側に寄ると service mesh、serialization、load balancing、deadline propagation の本になり、Kafka 側に寄ると broker、partition、consumer group、stream processing の本になりがちです。もちろんそれぞれ価値があります。しかし system を設計する立場から見ると、本当に欲しいのは個別技術の encyclopedic な説明ではなく、「いつ同期で頼み、いつ非同期に流し、失敗時に何が壊れるか」を一続きで考える地図です。

RPC は「相手に今すぐ仕事をしてほしい」という要請を表すのに向いています。一方 Kafka は、「今ここで完了させなくてもよいが、後で複数の側が読める形で残したい」という要求に向いています。両者は競合ではなく補完関係です。本書ではその補完関係を主題にします。

本書が扱う問い

本書では特に次の問いを中心に据えます。

- timeout が起きたとき、相手は本当に失敗したのか
- retry したとき、重複処理はどこで止めるべきか
- 同じ event を複数 service が読むとき、順序はどこまで守れるのか
- RPC の失敗を Kafka へ逃がせば本当に頑健になるのか
- queue や broker の長さは、単なる運用指標なのか、それとも設計上の信号なのか

これらの問いに共通しているのは、「相手の状態を完全には知らない」という点です。分散システムでは、失敗は単純な真偽値ではなく、観測できる failure と内部で起きている failure がずれることが普通です。そのずれをどう扱うかが、本書の中心線です。

この中心線を支える補助的な問いもあります。どの field を partition key にするべきか。どの request に idempotency key を持たせるべきか。consumer lag の増加は、単に worker を増やせばよいのか。どこからが user-facing critical path で、どこからが background work なのか。これらは別々の設問に見えますが、実際には「いつ結果を必要とし、何を一貫させ、失敗時に誰が責任を持つのか」という同じ問題へ戻っていきます。

対象読者と前提

読者には、TCP の基本、HTTP や gRPC の存在、queue や event の概念、Go か他の一般的な backend 言語で server を書いた経験があることを想定します。ただし分散システムの専門知識は前提にしません。Kafka を実運用したことがなくても構いません。本書では必要な概念を順に導入します。

一方で、初版では次を主題にしません。

- Kafka の storage engine や KRaft の細部

- service mesh の全体像
- CDC、Debezium、stream processing の全網羅
- Paxos / Raft の定理や formal proof
- multi-region database や geo-replication の深掘り

これらは重要ですが、最初の一冊で同時に扱うと中心線がぶれます。本書の初版は、サービス間通信の設計判断に必要な地図を作ることを優先します。

また、本書は製品マニュアルではありません。`gRPC` の flag 一覧、Kafka broker の内部 storage layout、Kubernetes の manifest 断片を大量に並べるより、設計時に何を先に決めるべきかへ焦点を当てます。だからコード例も本物の production 実装より短く、説明のための擬似コードに寄せます。目的はコピペ可能なサンプルを配ることではなく、読者が自分の system へ応用できる判断軸を手に入れることです。

この本の見方

本書では「同期通信は簡単、非同期通信は高度」といった序列は取りません。同期通信には timeout と cascading failure の難しさがあり、非同期通信には順序と重複と再処理の難しさがあります。難しさの種類が違うだけです。

したがって読むときの軸は、技術名ではなく次の観点です。

- いまの通信は request/response か、log への append か
- caller はいつ結果を必要としているか
- 失敗時に誰が retry し、誰が重複を吸収するか
- 順序や整合性はどの範囲で必要か
- 負荷が上がったとき、どこで backpressure をかけるか

> **注意**
> 「Kafka を入れれば非同期になって頑健になる」「`RPC` なら単純だから安全」といった理解は危険です。同期・非同期の違いは、難しさが消えることではなく、どの層へ移動するかです。本書では、その移動先を明確にします。

図やコードを見るときも、常に次の 3 つを自問すると理解しやすくなります。

1. この call / event の caller は、いつ結果を必要としているのか
2. 失敗したとき、どの層が retry し、どの層が重複を吸収するのか
3. 順序や整合性を守りたい単位は何か

この 3 問に答えられれば、多くの API や middleware は「よく分からない仕組み」ではなく、設計上の道具として見えてきます。

本書を通して繰り返す 4 つの問い

本書を読み進めると、`timeout`、`lag`、`idempotency`、`replay`、`ownership` といった言葉が何度も出てきます。これらは別々のトピックに見えますが、実際には次の 4 問へ戻っていきます。

1. caller はいつ結果を必要としているか
2. 成功を返したとき、何を保証したことにするのか
3. 重複や順序崩れはどこで吸収するのか
4. 壊れたとき、誰が観測し、誰が修復するのか

RPC の章では 1 と 2 が前に出ます。Kafka と consumer の章では 3 が前に出ます。playbook の章では 4 が前に出ます。しかし主題は同じです。読むときにこの 4 問へ戻ると、各章の論点がつながりやすくなります。

継続例で追う状態境界

本書の注文システムでは、同じ 注文 でも見る人によって境界が違います。

- client `accepted` / `rejected` / `temporary_failure`
- Order Service inventory と payment を含む同期 path の結果
- Kafka consumer `OrderCreated` をそれぞれの速度で処理する非同期 path
- operator lag、DLQ、redrive、manual recovery の対象

この境界を混ぜると会話が壊れます。たとえば 注文は成功している と 確認メールはまだ送られていない は両立しますし、`client` は `timeout` を見た と `backend` では遅れて `accepted` になった も両立します。分散システムでは、どの観測者から見た状態か を明示する癖が重要です。

```
Same order, different viewpoints

Client
  accepted / rejected / temporary_failure

Order Service
  inventory + payment + order row + outbox

Consumer side
  mail sent / analytics updated / search refreshed

Operator
  lag / DLQ / replay / recovery
```

この図式を頭に置くと、各章で出てくる 成功 や 失敗 が、誰の視点で語られているかを追いやすくなります。

accepted、rejected、unknown を分けて読む

分散設計の初学者がつまずきやすいのは、`success` と `failure` だけで世界を見てしまうことです。実務では、そのあいだに まだ断言できない 状態がかなりあります。

- `accepted` 注文は同期 path の意味で成立している
- `rejected` 業務的に成立していない
- `unknown` caller からはまだ断言できない。user-facing response では `temporary_failure` に近い

本書の中では、この `unknown` が重要です。payment timeout、response loss、relay 遅延、consumer lag は、どれも 何かが壊れた だけでなく 状態確認が必要になった と読むべき場面です。だから inquiry path、idempotency key、runbook が必要になります。

この 3 分類で読むと、同期 `RPC` と Kafka の役割も整理しやすくなります。`RPC` は `accepted` / `rejected` / `unknown` を早く狭めるための道具であり、Kafka は `accepted` のあとに残る仕事を時間方向へ押し出すための道具です。本文の user-facing な説明では、この `unknown` を多くの場合 `temporary_failure` として表現します。

各章の見取り図

本書は次の順で進みます。

1. 注文作成 の critical path を使って request/response の設計を整理する
2. RPC client 運用、load balancing、timeout budget、streaming を追加する
3. order-created event を使って Kafka の log モデルと producer 設計を見る
4. consumer group、lag、DLQ、redrive を通して worker 設計を見る
5. delivery semantics、ordering、outbox、saga を通して end-to-end consistency を考える
6. 最後に障害対応、観測、ownership、使い分けの playbook をまとめる
7. 付録で演習、用語、設計レビュー観点、次に読む資料を整理する

この順序には理由があります。Kafka から入ると、なぜ同期 request では足りないのかが見えにくくなります。逆に RPC だけ見ていると、結果を今すぐ返さなくてよい仕事や fan-out する event の扱いが抜けます。両方を見ることで、通信設計の判断軸が立体的になります。

各章では必ず継続例へ戻ります。たとえば Inventory Service はなぜ同期 RPC で扱うのか、Email Worker はなぜ Kafka consumer にするのか、Payment Service にはなぜ idempotency key が必要なのか、Search Indexer はなぜ replay を前提に設計すべきなのか、といった問いです。別々の service を見るのではなく、1 つの user 体験を支える部品として眺めることが重要です。

最初に押さえる読み方

読み進めるときは、全部を同じ深さで覚える必要はありません。まず次の順で押さえると入りやすいです。

1. 02 request/response が何を保証する契約か
2. 04 Kafka が queue ではなく log として何を増やすか
3. 05 consumer 側にどんな責務が移るか
4. 06 end-to-end で何が 1 回に見え、何が遅れてよいか
5. 07 壊れたときに何をどの順で見るか

この順で一周すると、本書の中心線である 同期 path で意味を固定し、非同期 path で副作用を分離し、障害時は ownership と runbook で回復する という流れが見えやすくなります。

この本を読み終えたときの出口

最終的に目指しているのは、Kafka を説明できる や gRPC を使える だけではありません。自分の system を見たときに、少なくとも次を言葉にできる状態です。

- この API の success は何を意味するか
- retry してよい境界と、してはいけない境界はどこか
- event の ordering scope は何か
- lag や DLQ を誰が見るべきか
- unknown state が起きたとき、どこで確認するか

そこまで行けば、新しい middleware や product に出会っても、その道具がどの問題を引き受け、どの問題を残すか で考えやすくなります。本書が作りたいのは、特定技術の暗記ではなく、この判断の地図です。

編集注記

本書は特定の framework の使い方本ではありません。gRPC、HTTP API gateway、Kafka producer/consumer のコード例は登場しますが、それらは API の暗記ではなく、「何を

caller の責務にし、何を broker や consumer の責務にするか」を考えるための足場として使います。

request/response と critical path

多くの system は、最初の一歩として request/response から始まります。user はボタンを押し、API server は依存先へ call し、結果を返します。構図としては単純です。しかし実務で本当に難しいのは、request/response そのものより、「どこまでを user-facing critical path に置くか」「そこで起きる失敗をどう user へ見せるか」です。この章では、注文作成フローを題材に、同期通信を設計するときの最初の判断軸を整理します。

この章の到達点

この章では次を押さえます。

- user-facing critical path をどう切り出すか
- data / temporal / failure contract をどう揃えるか
- timeout、deadline、retry をどこで決めるか
- idempotency key が必要になる理由
- response を返した時点で何を保証したと見なすか

critical path を先に決める

注文 API を設計するとき、多くの人は最初に「どの service を呼ぶ必要があるか」を考えます。しかしその前に決めるべきなのは、「何を今すぐ完了したことにしたいか」です。たとえば 注文を受け付けた と メール送信が終わった では、user にとっての重要度と時間制約が違います。これを区別しないと、なんでも同期 RPC でつないでしまい、遅延と failure surface だけが増えます。

注文作成フローを分解すると、典型的には次の 3 層があります。

- 今すぐ答えが必要なもの 在庫があるか、支払い方法が受け付けられるか、注文番号を返せるか
- 少し遅れてもよいが、確実にやりたいもの 確認メール、分析イベント、検索 index 更新
- 後で再実行してもよいもの BI への集計反映、recommendation feature の学習入力

この 3 層を混ぜると、checkout 画面の 1 request が不必要に長くなります。critical path の設計は、単なる performance tuning ではなく、user-visible state の定義です。

継続例: 注文作成の同期部分

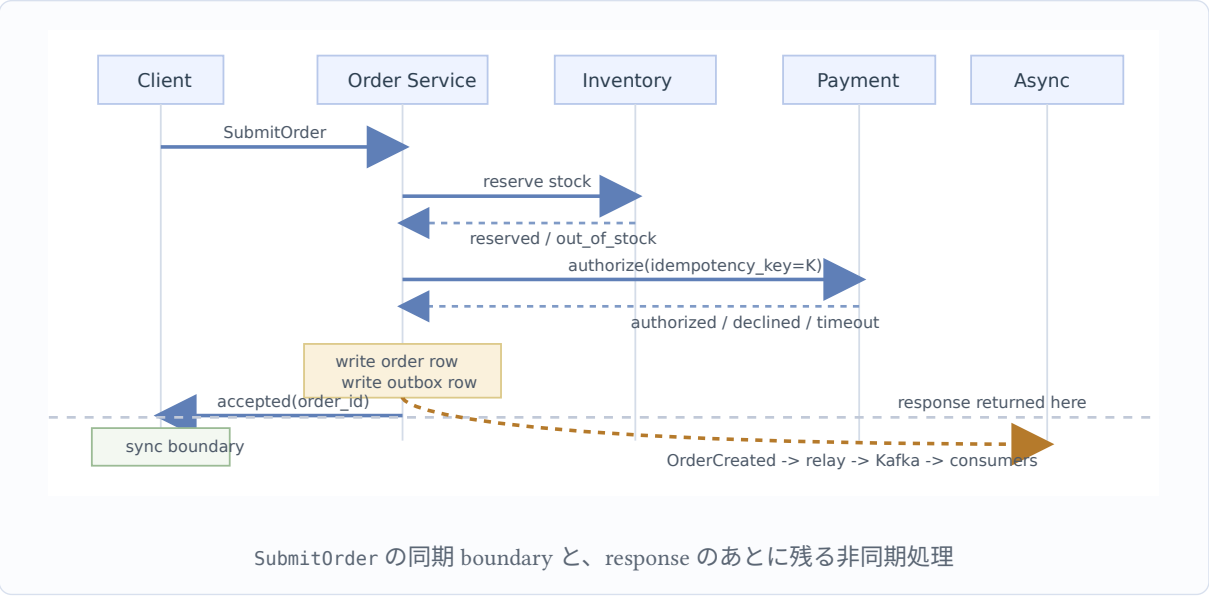
本書では、次のような注文作成フローを最小モデルとして使います。

```
Client
  -> API Gateway
    -> Order Service
      -> Inventory Service: reserve stock
      -> Payment Service: authorize payment
      -> Order DB: create order row
      -> Outbox Table: enqueue order-created event
  <- order accepted / failed
```

ここで critical path に入っているのは、少なくとも次の 4 つです。

1. request validation
2. 在庫確保
3. 支払いオーソリ
4. 注文 row の永続化

一方、メール送信や検索更新は入りません。これらは重要ですが、注文受理の瞬間に user が同期的に待つ理由は薄いからです。



`SubmitOrder` の同期 boundary と、response のあとに残る非同期処理

response が意味するものを固定する

注文 API が `200 OK` や `OrderAccepted` を返したとき、それは何を意味するのでしょうか。ここを曖昧にしたままでは、後続設計も曖昧になります。

最低でも次のどれかに落とす必要があります。

- 注文は確定し、在庫も支払いも押さえられた
- 注文は受け付けたが、後段処理はまだ進行中
- 注文リクエストは受理しただけで、確定可否は後で決まる

どれを選ぶかで同期部分の設計は変わります。EC の checkout なら、多くの場面では「支払いオーソリと在庫確保が済み、注文番号が発行された」ことまでを同期で保証したほうが自然です。逆に高トラフィックの業務システムでは、「受理のみ」で返して後段で審査する構成もあり得ます。大切なのは、response の意味を product と backend の両方で共有することです。

```
Response meaning comparison

1. accepted = order row + inventory reserved + payment authorized
   長所: user に説明しやすい
   短所: 同期 path が重い

2. accepted = order row only, downstream continues later
   長所: 速い
   短所: user-visible state が曖昧になりやすい

3. received = request was queued, final result later
```

長所: 極端な負荷に強い
短所: product 側の状態設計が難しい

継続例では 1 を採ります。理由は、注文直後に 在庫は押さえられているのか、支払いは通ったのか を user へ明確に返したいからです。副作用だけを非同期へ逃がし、注文確定そのものは同期 path に残します。

contract は 3 層ある

request/response を設計するとき、schema だけに注意が向きがちです。しかし同期通信の contract は少なくとも 3 層あります。

- data contract field 名、型、必須性、versioning
- temporal contract 期待される latency、deadline、stream の寿命
- failure contract retry 可否、idempotency、部分成功の扱い、error taxonomy

たとえば `ReserveStock(order_id, items)` という `RPC` を考えます。field が合っていても、temporal contract が不明なら caller は何秒待つべきか分かりません。failure contract が不明なら、timeout 時に再送してよいのか分かりません。分散設計の難しさは、多くの場合 data contract 以外にあります。

data contract

注文システムでは、識別子の扱いがとくに重要です。`order_id`、`customer_id`、`payment_attempt_id`、`idempotency_key` の役割が混ざると、あとで重複や整合性で苦しくなります。`order_id` は業務上の注文を表し、`payment_attempt_id` は支払い試行を表し、`idempotency_key` は重複 request の吸収に使う、といった責務分離を最初からしておくと後が楽です。

temporal contract

checkout API 全体に 1.5 秒の deadline を持たせるなら、`Inventory Service` と `Payment Service` に同じ 1.5 秒をそのまま配るべきではありません。validation、DB 書き込み、serialization、network jitter にも時間が要るからです。したがって、親 request の budget を子 call に配分する設計が必要です。

failure contract

`Inventory Service` が `OUT_OF_STOCK` を返したときと、`UNAVAILABLE` を返したときでは意味が違います。前者は user に在庫切れを見せるべき失敗であり、後者は一時的障害かもしれません。同じ `error` でも retry 可否が違う以上、failure contract を型や status として明示する必要があります。

timeout と deadline を予算として扱う

分散システムで timeout を設計するとき、「何秒にするか」を 1 つ決めれば終わりではありません。重要なのは、request 全体の budget をどの段にどう割り当てるかです。

たとえば checkout API 全体の SLO を 1.5 秒とします。おおまかに次のような配分が考えられます。

- request validation と認証: 100ms
- Inventory Service 呼び出し: 250ms
- Payment Service 呼び出し: 400ms
- DB transaction: 250ms
- serialization と network 余裕: 200ms

- retry / jitter / buffer: 300ms

この配分は固定値そのものが大事なのではなく、`無限に待たない` ことと `子 call が親 deadline を尊重する` ことが大切です。親 request がすでに失敗扱いになったあとも downstream が作業を続けると、resource を浪費し、結果の意味も曖昧になります。

```go
func (h *CheckoutHandler) SubmitOrder(ctx context.Context, req *SubmitOrderRequest)
(*SubmitOrderResponse, error) {
    ctx, cancel := context.WithTimeout(ctx, 1500*time.Millisecond)
    defer cancel()

    invCtx, invCancel := context.WithTimeout(ctx, 250*time.Millisecond)
    defer invCancel()
    reservation, err := h.inventory.Reserve(invCtx, ReserveStockRequest{
        OrderID: req.OrderID,
        Items:   req.Items,
    })
    if err != nil {
        return nil, mapInventoryError(err)
    }

    payCtx, payCancel := context.WithTimeout(ctx, 400*time.Millisecond)
    defer payCancel()
    payment, err := h.payment.Authorize(payCtx, AuthorizeRequest{
        OrderID:        req.OrderID,
        Amount:         req.TotalAmount,
        IdempotencyKey: req.IdempotencyKey,
    })
    if err != nil {
        _ = h.inventory.ReleaseBestEffort(ctx, reservation.ReservationID)
        return nil, mapPaymentError(err)
    }

    order, err := h.repo.CreateOrderWithOutbox(ctx, req, reservation, payment)
    if err != nil {
        _ = h.payment.CancelBestEffort(ctx, payment.AuthorizationID)
        _ = h.inventory.ReleaseBestEffort(ctx, reservation.ReservationID)
        return nil, err
    }
    return &SubmitOrderResponse{OrderID: order.ID, Status: "accepted"}, nil
}
```

この擬似コードで重要なのは、Go の細部ではありません。`親 deadline がある、子 call へ budget を切る、失敗時に次の挙動を決めている` という設計意図です。

4 つの通しケース

ここまでの議論を、同じ `SubmitOrder` API の 4 つの結果として並べると違いが見えやすくなります。

ケース 1: 正常系

```
Client -> Order Service
  -> Inventory reserve ok
  -> Payment authorize ok
  -> DB tx ok
<- accepted(order_id=01)
```

このケースでは、response の意味は比較的明快です。問題は、ここだけを見て設計してしまうことです。本当に難しいのは次の 3 ケースです。

ケース 2: 在庫切れ

```
Client -> Order Service
  -> Inventory reserve => OUT_OF_STOCK
<- rejected(out_of_stock)
```

これは business failure です。retry や circuit breaker の対象ではなく、user-facing な結果として返すべきです。だから `500` や `INTERNAL` にしてはいけません。

ケース 3: 支払い timeout

```
Client -> Order Service
  -> Inventory reserve ok
  -> Payment authorize => timeout
  -> Inventory release best effort
<- temporary_failure
```

ここで caller が知っているのは `timeout` だけです。payment が未実行か、実行済みだが response を失ったのかは分かりません。したがって automatic retry を入れるなら idempotency key が必須ですし、補償や後追い確認の設計も必要になります。

ケース 4: response loss 後の client retry

```
Client -> Order Service
  -> Inventory reserve ok
  -> Payment authorize ok
  -> DB tx ok
  X response lost
Client retries same request
  -> Order Service sees same request identity
  -> returns prior result
```

このケースは実務で頻出です。backend 側から見ると最初の注文は成功しているのに、client は失敗したと思って再送します。ここで stable request identity がないと二重注文になります。

ケース 5: caller は諦めたが backend は遅れて成功した

```
Client -> Order Service
  -> Inventory reserve ok
  -> Payment authorize slow but eventually ok
Client timeout / user leaves page
  -> DB tx ok
  -> response cannot be delivered
```

このケースでは、user は `失敗したかもしれない` と感じていますが、backend では注文が既に成立しているかもしれません。だから request/response 設計では、`失敗したかもしれない request の後追い確認をどうするか` まで含める必要があります。

単に client へ `もう一度送ってください` と案内すると、二重注文や二重課金の危険を広げます。分散環境では `caller が失敗を観測した` と `業務操作が失敗した` は同じではありません。

stable request identity をどこで作るか

二重送信や response loss を吸収するには、request identity が stable でなければなりません。重要なのは、`同じ intent の再送` を同じ request と認識できることです。

選択肢は大きく 2 つあります。

- client が生成する network retry や page reload をまたいで同じ key を再利用しやすい
- server が最初の request で発行する 初回送信前の再試行には弱い

注文のように user の再送が起こりやすい操作では、client もしくは gateway 境界で stable key を持つ設計のほうが強いです。大切なのは `order_id` と混同しないことです。`order_id` は成功後の業務 ID であり、request identity は成功前から存在しなければ再送吸収に使えません。

```
Request identity choices

client_request_id
  用途: 同じ intent の再送吸収
  生存期間: 送信前から response 確認まで

order_id
  用途: 成立した注文の業務識別
  生存期間: 成功後

payment_idempotency_key
  用途: payment attempt の重複吸収
  生存期間: provider の idempotency window に依存
```

status inquiry を設計に含める

同期 API を設計するとき、意外に重要なのが 結果をあとから問い合わせる道 です。timeout や response loss が起きる以上、caller が この `request` は最終的にどうなったか を確認できるほうが安全です。

継続例なら、たとえば `client_request_id` で注文状態を引ける read API があると運用しやすくなります。

- `accepted` 注文は成立している
- `rejected` 業務的に成立していない
- `unknown / still processing` timeout 直後など、まだ確定していない

この問い合わせ経路があると、client 側も 失敗したら再送 一択ではなくなります。結果確認と再送を分けられるからです。request/response の設計は、最初の 1 回の response だけでは閉じません。

retry してよい call と、してはいけない call

retry は availability を上げるための道具ですが、何でも retry してよいわけではありません。分類の基本は次の通りです。

- 同じ request を再送しても安全な call 読み取り、自然に冪等な更新、idempotency key つきの create
- retry する前に影響範囲を考える call inventory reservation、payment authorization のように外部 state を変える call
- 原則として自動 retry を避ける call 明示的な side effect が大きく、重複時の影響が高い call

ここでよくある失敗は、「timeout したから未実行だろう」と思い込むことです。実際には callee が成功し、response だけが失われた可能性があります。したがって timeout 後 retry を許すなら、callee 側で idempotency を受け止める設計が必要です。

idempotency key は誰のためにあるのか

idempotency key は transport の都合ではなく、業務の正しさのためにあります。checkout 画面で user が二度押しした、mobile app が切断後に再送した、API gateway が upstream timeout を見て再試行した、といったときに、`Payment Service` が毎回新しいオーソリを切ってしまうと被害が大きいからです。

典型的には、`Order Service` が注文 request ごとに stable な idempotency key を持ち、それを `Payment Service` へ渡します。`Payment Service` はその key を見て、同じ key の過去結果を返すか、新規処理を 1 回だけ実行します。

```
type AuthorizeRequest struct {
    OrderID        string
    Amount         int64
    CustomerID     string
    IdempotencyKey string
}

func (s *PaymentService) Authorize(ctx context.Context, req AuthorizeRequest)
(Authorization, error) {
    if prev, ok := s.store.LookupByIdempotencyKey(req.IdempotencyKey); ok {
        return prev, nil
    }
    auth, err := s.gateway.Authorize(ctx, req)
    if err != nil {
        return Authorization{}, err
    }
    s.store.Save(req.IdempotencyKey, auth)
    return auth, nil
}
```

ここで重要なのは、`IdempotencyKey` が request の identity を表し、`OrderID` とは別の責務を持っていることです。Order 自体はまだ作れていなくても、支払い試行の重複吸収には stable な key が必要になります。

user-facing state は 3 種類に寄せる

注文 API の response を設計するとき、user-facing state を増やしすぎると product と client 実装が苦しくなります。同期 path の初期段階では、次の 3 種類に寄せると整理しやすいです。

- `accepted` 注文は成立した
- `rejected` 在庫切れや支払い拒否など、業務的に成立しない
- `temporary_failure` 技術的理由でいまは確定できない

もちろん内部ではもっと多くの error taxonomy を持ってよいですが、user-facing にはこの 3 つ程度に畳むと説明しやすくなります。大切なのは、`temporary_failure` を `unknown` に近い状態として扱い、安易に再送だけを促さないことです。

user に見せる失敗を設計する

同期 API では、技術的な失敗をそのまま user-facing error にしてはいけません。たとえば `Inventory Service` の `DEADLINE_EXCEEDED` は、user から見れば「いま在庫がない」ではなく、「一時的に注文処理に失敗した」に近いはずです。逆に `OUT_OF_STOCK` は明確な業務失敗なので、ただの `500` として返すべきではありません。

設計上のコツは、technical error と business error を分けることです。

- business error 在庫切れ、支払い拒否、入力不正
- transient technical error timeout、一時的な network failure、rate limit
- persistent technical error schema 不整合、プログラムバグ、誤設定

この区別を API 設計に入れておくと、retry policy と user メッセージを分けやすくなります。読者が実務で迷いやすいのは、ここを全部 `INTERNAL` や `500` にしてしまうことです。

> **編集注記**
> 分散システムでは、`error code` は transport detail であると同時に設計文書でもあります。retry 可否や user-facing state を左右するので、status 設計を「あとで決める」にしてはいけません。

支払いと在庫の順序をどうするか

注文作成の同期パスでは、`在庫確保を先にするか` と `支払いオーソリを先にするか` で議論が分かれます。どちらが絶対に正しいわけではありませんが、違いはあります。

- 在庫先行 品切れ時に無駄な payment call を減らせる
- 支払い先行 支払い拒否が多い業務なら inventory load を減らせる

どちらを選んでも、片方成功後にもう片方が失敗する可能性は残ります。したがって重要なのは「順序」そのものより、「失敗したときにどう解放・取消するか」を決めることです。同期 request の設計は、成功パスより失敗パスのほうが難しいのです。

```
Ordering comparison

Inventory first
  + 品切れ時に payment 呼び出しを避けやすい
  - payment timeout 時に reservation 解放が必要

Payment first
  + 支払い拒否が多い業務では inventory load を減らせる
  - inventory 失敗時に payment 取消が必要
```

継続例では inventory first を基本線にします。理由は `在庫がない注文に支払いオーソリを掛けない` ほうが product 的に自然だからです。ただし業務によっては逆もあり得るので、ここは本質的に business choice です。

response を返したあとに何が残るか

`SubmitOrder` が成功 response を返した時点でも、system 全体ではまだやるべきことが残っています。メール送信、analytics、検索更新、fraud scoring などです。ここで誤解してはいけないのは、`response を返した` と `全ての副作用が終わった` は別だということです。

逆に言うと、同期 path では「どこまで終わっていれば user に返してよいか」を厳密に決める必要があります。本書の継続例では、少なくとも次を同期で保証したことにします。

- 注文 row は永続化された
- 在庫確保または在庫切れ判定が済んだ
- 支払いオーソリまたは支払い拒否判定が済んだ
- 後続 event の発行要求は outbox に記録された

この最後の項目が重要です。Kafka に publish し終わっている必要まではありませんが、少なくとも「publish すべき事実」は失われない形で記録しておく必要があります。

API 名と状態名をずらさない

同期 API を設計するとき、request 名と業務状態名がずれていると混乱しやすくなります。たとえば API 名は `CreateOrder` なのに、実際の意味は `AcceptOrderRequest` に近い、ということがあります。これは product と backend の会話を崩します。

継続例では、次のような naming が自然です。

- `SubmitOrder` user からの注文要求を受ける API
- `OrderAccepted` 同期 path が成功し、注文番号が確定した状態
- `OrderCreated` Kafka に流す事実としての event 名

この 3 つは似ていますが同じではありません。`SubmitOrder` は request、`OrderAccepted` は synchronous response の意味、`OrderCreated` は後続 consumer が読む log 上の事実です。名前を揃えることは、状態境界を揃えることでもあります。

read-your-own-writes をどこまで求めるか

注文作成 API のあとに user がすぐ注文一覧画面を開くとします。このとき user は「さっきの注文が見えるはずだ」と期待します。これは read-your-own-writes の要件です。

同期 path で orders table への書き込みが終わっていれば、少なくとも Order Service の read model では見えるようにできます。しかし検索 index や analytics dashboard まで即時に見える必要があるかは別問題です。ここを分けて説明できないと、全 projection を同期化したくなってしまいます。

継続例なら、次の整理が自然です。

- 注文詳細画面 同期 DB 書き込みに基づくので、すぐ見えるべき
- 検索結果 数秒遅れても許容できるなら非同期 index 更新でよい
- 管理画面の売上集計 数分遅れでもよいなら analytics consumer でよい

つまり read-your-own-writes もスコープを切る必要があります。

failure mapping の小さな表

設計時に便利なのは、「どの失敗を user にどう見せるか」を表にしてしまうことです。文章だけで持つと、後から retry policy と食い違いやすくなります。

```
Inventory: OUT_OF_STOCK
  -> user-facing: 在庫切れ
  -> retry: no

Inventory: DEADLINE_EXCEEDED
  -> user-facing: 一時的に処理できません
  -> retry: caller may retry once

Payment: CARD_DECLINED
  -> user-facing: 支払い方法を確認してください
  -> retry: no automatic retry

Payment: UNAVAILABLE
  -> user-facing: 一時的に処理できません
  -> retry: only with idempotency key
```

このレベルの表があるだけで、実装時に `全部 INTERNAL` に倒す事故が減ります。

caller cancellation を無視しない

mobile app や browser では、user が画面を閉じたり、通信が切断されたりすることがあります。すると upstream は request を諦めます。ここで downstream がその事実を知らず、inventory や payment を延々と続けると、user が既に離脱した request のために資源を使い続けることになります。

したがって同期 path では、caller cancellation を子 call へ伝播させることが重要です。Go の `context` はそのための道具ですし、他言語でも cancellation token や deadline propagation の仕組みがあります。

ただし cancellation を伝えれば全て安全になるわけではありません。`Payment Service` が既に外部 gateway へ request を投げているなら、途中で caller が諦めても処理は止まらないかもしれません。この場合も、あとで問い合わせ・取消・重複吸収が必要です。だから cancellation は「節約」の仕組みであり、「実行されなかった保証」ではありません。

小さな通しシナリオ: 二重押しからの回復

最後に、実務でかなり多い 二重押し を短く通しておきます。

```
T0 user clicks "Place order"
T1 client sends SubmitOrder(client_request_id=R1)
T2 payment is slow, page spinner continues
T3 user clicks again or app retries
T4 client sends SubmitOrder(client_request_id=R1) again
T5 Order Service recognizes same request identity
T6 returns prior accepted result or current processing status
```

この流れで守りたいのは、同じ意図を複数回実行しない ことです。request identity、payment idempotency、status inquiry が揃っていると、user の再送を事故ではなく通常系として扱えます。同期設計の成熟度は、このような地味なケースにどれだけ耐えられるかで分かります。

critical path から外す基準

設計レビューで迷いやすいのは、「この処理は同期 path に残すべきか」です。継続例なら次の基準が使えます。

- 注文確定の意味を作る処理 同期に残す
- user-facing response に即座に影響しない副作用 非同期へ逃がす
- 再実行や replay が自然な処理 非同期へ逃がしやすい
- その場で user に見せる必要がある validation 同期に残す

これを箇条書きの感覚で終わらせず、実際の処理を棚卸しするのが重要です。たとえば fraud scoring は business によって分かれます。注文を即確定したいなら非同期 review が自然ですし、高リスク商材なら同期 path に残すかもしれません。

章末まとめ

この章の本質は、request/response を 関数呼び出しの分散版 として見るのをやめることです。同期通信は、critical path と response meaning を定義するための契約です。その契約を支えるのが timeout budget、failure mapping、idempotency、cancellation propagation です。

注文システムで見れば、`SubmitOrder` が成功した瞬間に何を保証するかを決めることが、後続の Kafka 設計や consumer 設計の前提になります。ここが曖昧だと、後ろの章をどれだけ丁寧に書いても全体がぶれます。

この章の設計判断の要点

同期通信の設計で最初に見るべきなのは、HTTP method や `proto` の形ではありません。critical path、response の意味、timeout budget、failure contract です。これが決まっていれば、実装技術が `gRPC` でも HTTP/JSON でも本質はあまり変わりません。

言い換えると、request/response は「相手を呼ぶ技術」ではなく、「いま結果が必要な仕事を扱う契約」です。だからこそ retry、idempotency、error mapping を最初から同時に考える必要があります。

> **注意**
> 同期通信だから簡単 という感覚は危険です。実務で壊れやすいのは、たいてい成功パスではなく、timeout、再送、部分失敗、重複実行が絡む場面です。

演習

1. あなたの system で、user-facing critical path に入っている処理を 5 つ挙げ、それぞれを「本当に今すぐ必要か」で見直してください。
2. `SubmitOrder` API が `accepted` を返した時点で、何を保証し、何はまだ保証していないかを 3 行で定義してください。
3. inventory timeout 時に caller が自動 retry してよい条件を整理してください。
4. payment authorization に idempotency key がない場合、どの failure で二重課金が起き得るかを書き出してください。
5. 画面リロードや mobile app 再送を前提にしたとき、stable request identity をどこで生成すべきかを設計してください。
6. あなたの system で、同期 path に残っているが実は非同期へ逃がせそうな処理を 3 つ挙げてください。

この章の出口

request/response を critical path と user-facing state の契約として読めるようになると、次に必要なのは「その契約を本番でどう維持するか」です。次章では load balancing、connection 管理、streaming、observability を加えて、同期通信を運用対象として見ます。

RPC の運用設計

前章では request/response を critical path の設計として見ました。しかし本番 system で同期通信を扱うには、それだけでは足りません。client はどの instance へつなぐのか、timeout は hop ごとにどう配るのか、retry はどこまで許すのか、connection pool はどのくらい持つのか、streaming をいつ使うのか、といった運用の論点が必ず出てきます。この章では、注文システムの `Inventory Service` と `Payment Service` を題材に、`RPC` を運用可能にするための設計を整理します。

> **この章の到達点**
> この章では次を押さえます。
>
> - service discovery と load balancing の役割
> - connection pool と queueing が latency をどう変えるか
> - retry budget、circuit breaker、rate limit の意味
> - unary と streaming をどう使い分けるか

- tracing / metrics で critical path をどう観測するか

論理名で呼ぶということ

`Order Service` は実際には `inventory-service-7f4d9b` のような pod 名を知りません。知っているのは `inventory-service` という論理名だけです。そこから先は service discovery と load balancing の仕事です。

この一見地味な仕組みが重要なのは、分散システムでは「相手」は固定 1 台ではないからです。deploy によって instance は増減し、rolling restart が走り、一部 instance だけが unhealthy になることもあります。`RPC` client は単なる library call ではなく、動的に変わる接続先集合へ request を流す component です。

設計上は、次の問いを持つと理解しやすくなります。

- 接続先一覧は誰が更新するのか
- unhealthy instance を誰が外すのか
- retry 時に同じ instance へ行くのか、別 instance へ送るのか
- region / zone / cell をまたぐとき、どこまで local preference を持たせるのか

load balancing は throughput だけの話ではない

load balancing と聞くと、多くの人は「負荷を均等にすること」と理解します。もちろんそれも大切ですが、実務ではそれ以上の役割があります。

- skew を減らし、一部 instance だけが詰まるのを避ける
- rolling deploy 中に traffic を滑らかに移す
- 障害 instance を切り離す
- zone 障害時に traffic を逃がす

つまり load balancing は性能最適化というより、failure containment の仕組みでもあります。

ただし load balancing は万能ではありません。retry と組み合わさると、もともと 1 回で済んだ request が複数 instance に飛び、重複実行の面積が広がることもあります。特に stateful な side effect を持つ `Payment Service` では、この点を軽視できません。

どの balancing policy を選ぶか

単純な round-robin は分かりやすいですが、queue が長い instance へも均等に投げてしまうことがあります。一方 least-loaded 系は理論上よさそうでも、観測の遅れや load estimate の難しさがあります。

実務で大切なのは、policy 名の暗記より、`局所的に速い instance を優先しすぎて偏らないか、失敗 instance をどれくらい早く避けられるか` を考えることです。最初の一冊としては、次の程度の理解で十分です。

- round-robin 単純だが、queue length を見ない
- random / power of two choices 実装が比較的簡単で偏りを減らしやすい
- request-aware policy latency や in-flight 数を見て選ぶが、複雑になりやすい

system の初期段階では単純な policy でも構いません。重要なのは policy を固定することではなく、tail latency と error rate を見て変えられることです。

connection pool は invisible queue でもある

RPC client library は多くの場合、下に connection pool を持ちます。これは便利ですが、同時に invisible queue にもなります。pool size が小さすぎると request は socket 待ちで詰まり、大きすぎると server 側の queue や kernel resource を圧迫します。

つまり caller の latency は、callee の実処理だけでなく、次の 3 つの待ち時間の合計です。

- caller 側 queue / pool 待ち
- network 往復
- callee 側 queue / 実行待ち

多くのトラブルは、真ん中の network ではなく、両端の queue にあります。`Inventory Service` が遅いように見えて、実際には caller 側で connection 待ちが起きていた、というのはよくある話です。

```
caller goroutine
  -> client-side queue
    -> connection pool acquire
      -> network
        -> server listener queue
          -> server worker queue
            -> handler
```

この図を見ると分かるように、`RPC latency` は 1 つの箱の時間ではありません。複数 queue の足し算です。

timeout budget と retry budget

前章で timeout budget を見ましたが、運用では retry budget も同じくらい重要です。retry budget とは、「失敗時にどのくらいまで追加試行を許すか」という上限です。これがないと、障害時に request が雪だるま式に増えます。

注文システムなら、たとえば次のような設計が考えられます。

- `Inventory Service` 小さな timeout、1 回まで retry、同一 request 内で total 250ms を超えない
- `Payment Service` 自動 retry はかなり保守的、idempotency key 前提、gateway 側 failure taxonomy を強く見る

重要なのは、全 hop が独立に 3 回 retry するような構成を避けることです。API Gateway、Order Service、client library、sidecar が全部 retry すると、障害時の amplification は簡単に数倍から数十倍になります。

```
Checkout path timeout sketch

Client -> API Gateway                 total budget 1500ms
API Gateway -> Order Service         1400ms
Order Service -> Inventory Service    250ms
Order Service -> Payment Service      400ms
Order Service -> DB transaction       250ms
Leftover for retry / jitter / write   300ms
```

このように budget を明示しておくと、「どこが勝手に待ちすぎているか」を議論しやすくなります。分散システムの timeout は気分で決める数字ではなく、全体制約の配分です。

circuit breaker は何を守るのか

circuit breaker は fashionable な pattern ですが、本質は単純です。すでに失敗率が高く、短期的に回復しそうにない依存先に対して、これ以上 caller 資源を浪費しないようにする仕組みです。

breaker を入れる場所で大切なのは、「誰を守りたいのか」を明確にすることです。

- caller を守る goroutine、thread、connection、memory の枯渇を防ぐ
- callee を守る 既に苦しい依存先へ無駄な retry を積まない
- user を守る 無限に spinning せず、早めに degraded response を返す

breaker が開いたときに何を返すかも設計が要ります。たとえば recommendation service なら degraded でもよいかもしれませんが、payment authorization では単に `success` に倒すわけにはいきません。

rate limit と concurrency limit

`RPC` の障害は、相手が遅いときだけ起きるわけではありません。トラフィックの急増でも起きます。ここで効くのが rate limit と concurrency limit です。

- rate limit 単位時間あたりの request 数を制限する
- concurrency limit 同時に処理中の request 数を制限する

前者は flood を防ぎやすく、後者は queue の伸びを防ぎやすいです。多くの system では両方を組み合わせます。`Inventory Service` が在庫 DB に重い lock を取りやすいなら concurrency limit が効きますし、public API の abusive traffic を抑えるなら rate limit が効きます。

> **編集注記**
> 分散システムで「全部受けてから中で頑張る」は危険です。受け入れの境界で拒否する設計を持たないと、遅い request が queue を膨らませ、正常 request まで巻き添えにします。

unary と streaming の使い分け

`RPC` というと unary request/response を思い浮かべがちですが、streaming は意外に重要です。注文システムでも、たとえば `Payment Service` と fraud engine のあいだで進捗付き審査を流したい場合や、`Order Service` から backoffice へ大量注文の status を順次流したい場合、streaming が向きます。

ただし streaming を使う理由は「かっこいいから」ではありません。主な利点は次の通りです。

- per-RPC の HTTP/2 ストリーム開始コスト（HEADERS frame）を複数メッセージで分散できる
- 部分結果や進捗を返せる
- 高頻度更新を 1 stream にまとめられる

一方で streaming は session lifetime の管理、flow control、片側 failure の扱いが難しくなります。過去 event を後から追いかけて読みたいなら、streaming ではなく Kafka のような log が向きます。

streaming は queue の代わりではない

ここは誤解しやすい点です。bidirectional stream があると「Kafka を使わず stream で流せるのでは」と思いがちですが、役割が違います。

- streaming 生きている session 間で、いま発生している data を流す
- log / broker session をまたいで、後からでも読める形で data を残す

たとえば `Analytics Consumer` が 3 時間停止していたあとに過去 event を読み直したいなら、streaming は向きません。Kafka はこの用途のためにあります。

tracing と metrics は一緒に設計する

`RPC` の observability で最初に必要なのは、`p95 latency` だけではありません。少なくとも次を揃える必要があります。

- success rate / error rate
- p50 / p95 / p99 latency
- timeout rate
- retry rate
- client-side queue wait
- server-side queue wait
- in-flight request 数

さらに tracing では、1 request が `Inventory Service`、`Payment Service`、DB にどの順で何 ms 使っているかを見えるようにします。checkout の tail latency を縮めるとき、どの hop が原因か分からなければ対処できません。

```
trace: submit-order
  authn                12ms
  inventory.reserve    48ms
  payment.authorize   221ms
  db.tx                85ms
  write response       10ms
```

この程度の時系列が見えるだけでも、改善の方向性はかなり定まります。

監視設計を先に持つ

設計段階で忘れやすいのが、「どの metric があればこの call chain を診断できるか」です。checkout path のような重要経路では、少なくとも次の組み合わせが必要です。

- `submit-order` 全体の p95/p99 latency
- `inventory.reserve` と `payment.authorize` の success/error/timeout
- client-side queue wait
- in-flight request 数
- retry count

これらが揃っていないと、遅い という報告が来ても、network なのか pool なのか downstream なのかを切り分けにくくなります。観測は実装後の飾りではなく、運用可能性の一部です。

deadline を hop ごとに伝える

timeout を各 service が勝手に持つだけでは、distributed call chain は安定しません。重要なのは、request 全体の deadline を hop ごとに伝え、各 hop が 残り時間 を見て振る舞うことです。

たとえば `submit-order` に 1500ms の overall deadline があるなら、`Order Service` は自分の手元時間を差し引いたうえで `Inventory Service` と `Payment Service` に短い budget を渡すべきです。これをしないと、上流は残り 200ms しかないのに、下流は 1 秒待つ、といった無意味な call が起きます。

```
Deadline propagation example

Client deadline          t0 + 1500ms
API Gateway local work   80ms
Order Service receives   remaining 1420ms
Inventory call budget    250ms
Payment call budget      400ms
DB + response reserve    350ms
Jitter / retry reserve   420ms
```

この設計の価値は、速くすることだけではありません。どうせ成功しない request を早く諦め、queue と connection を守ることにあります。deadline は user 体験のためだけでなく、system 自身の保護装置でもあります。

tail latency と hedged request

`RPC` の世界では、平均より tail latency のほうが user 体験に効くことが多いです。1 台だけ遅い instance、1 回だけ重い GC、1 回だけ cold connection が混ざるだけで p99 は大きく悪化します。

一部の read-heavy system では、hedged request、つまり遅い call を見て別 instance へ予備 call を出す手法が使われます。ただし本書の継続例のように side effect を持つ call、特に payment や inventory reservation では慎重であるべきです。idempotency が強く担保されていないなら、hedging は重複実行の危険を広げます。

つまり tail latency 対策も、`読み取り` と `副作用つき書き込み` で同じではありません。

service mesh が解決することと、しないこと

実務では `RPC` の運用に service mesh や共通 sidecar が入ることがあります。これにより mTLS、service discovery、retry、telemetry を統一しやすくなります。これは確かに便利です。

しかし mesh があっても、次の設計は依然として application 側の責務です。

- どの call を retry してよいか
- どの error を business error と見るか
- idempotency key をどう持つか
- degraded mode を何にするか

mesh は transport を助けますが、業務意味までは解釈しません。ここを誤解すると、「retry は mesh に任せたから安全」と思い込んでしまいます。

deploy 時の draining を軽視しない

同期通信では、普段の steady-state より deploy 時に事故が起きやすいことがあります。rolling update で instance を入れ替えるとき、古い instance への connection をどう drain するかを考えていないと、短時間の 5xx や timeout が急増します。

とくに注意が必要なのは次の場面です。

- keepalive 接続が古い instance に張りついたままになる
- long-lived stream が切断され、caller が大量再接続する

- readiness は落ちているが、in-flight request がまだ残っている
- autoscaling と rollout が重なり、rebalance 的な揺れが起きる

この種の障害は `business logic が壊れた` のではなく、通信面のライフサイクル設計が弱いことから起きます。同期 path が重要なほど、deploy は単なる配布作業ではなく通信イベントでもある、と見たほうがよいです。

cache と request coalescing

すべてを `RPC` で毎回取りに行く必要はありません。とくに read-heavy な依存先では、短い TTL cache や request coalescing が効くことがあります。たとえば商品メタデータや配送オプションのように、checkout で参照するが秒単位で変わらない情報です。

ただし cache を入れるときも、何を stale にしてよいかを定義する必要があります。inventory の残数や payment authorization の結果のように、鮮度が本質なものには雑な cache は危険です。逆に reference data には有効です。

request coalescing も同様で、同じ key への同時 request を 1 つにまとめることで downstream load を減らせますが、timeout 共有や head-of-line blocking の形で新しい難しさも入ります。結局ここでも、「何を共有してよいか」という設計判断が先にあります。

rollout と互換性

同期 `RPC` は、schema 変更の影響が比較的すぐに表面化します。したがって versioning と rollout の手順が重要です。基本は次の通りです。

- callee を先に前方互換にする
- caller をあとから切り替える
- 不要 field の削除は最後にする
- error contract の変更は特に慎重にする

とくに failure contract の互換性は見落とされがちです。新しい caller が `RETRYABLE_UNAVAILABLE` を期待しているのに、旧 callee は全部 `INTERNAL` で返す、といったズレがあると retry policy が壊れます。

call graph を短く保つ

実務では、`RPC` の問題の多くは 1 本の call の難しさではなく、call graph が長くなりすぎることから来ます。API Gateway -> Order Service -> Inventory Service -> Pricing Service -> Feature Flag Service -> User Segment Service のように深くなると、latency も failure surface も急増します。

checkout のような critical path では、次の原則が効きます。

- 依存先を増やしすぎない
- 同じ情報を複数 hop で取り直さない
- 必須でない副作用は同期 path から外す
- 失敗時に degraded でよい依存先を切り分ける

call graph を短くすることは、美学ではなく可用性の設計です。

典型的な設定ミス

`RPC` が遅い、壊れやすい、と感じる system の多くは、理論より先に基本設定で失敗しています。よくあるのは次です。

- timeout が無い、または長すぎる
- caller と sidecar と gateway が多重に retry する
- connection pool が飽和しているのに見えていない
- business error と technical error が同じ status に潰れている
- 全依存先に同じ retry policy を使っている

この一覧は地味ですが、実務では非常に効きます。難しい pattern を導入する前に、まずここを潰すべきです。

ケーススタディ 1: payment timeout が急増した日

checkout の p99 が急に悪化し、`payment.authorize` の timeout が増えたとします。このとき重要なのは、`payment が遅い` とだけ見ないことです。実際には少なくとも次の可能性があります。

- `Payment Service` 自身が遅い
- caller 側の connection pool acquire が詰まっている
- timeout が短すぎて、成功直前で切っている
- retry が多重化して、障害を増幅している

診断の順番としては次が分かりやすいです。

1. `payment.authorize` の success / error / timeout を見る
2. client-side queue wait と pool saturation を見る
3. retry count が跳ねていないかを見る
4. breaker が開いているかを見る
5. 直近 deploy / config change を確認する

この順で見れば、`network か payment か` という粗い分類より、`どこで待ち時間が積まれているか` へ近づけます。

ケーススタディ 2: inventory p99 は悪いが service 自体は healthy

`Inventory Service` 側の CPU も DB も平常なのに、`Order Service` から見ると inventory call の p99 だけが悪いことがあります。こういうときは server より caller 側を疑う価値があります。

典型的には次の形です。

```
submit-order trace
  client-side queue wait    140ms
  network                     4ms
  inventory handler          18ms
```

この場合、原因は `Inventory Service` ではなく、caller 側で接続が取れず queue していることです。同期通信の運用では、`遅い RPC` を `遅い callee` と短絡しないことが重要です。

dependency ごとに予算表を持つ

checkout のような重要経路では、依存先ごとの `時間予算` と `失敗時の扱い` を小さな表にしておくと、レビューしやすくなります。

```
Checkout dependency table

Inventory Service
  timeout: 250ms
  retry: 1
```

```
  idempotency: reservation_id based release
  degraded: no

Payment Service
  timeout: 400ms
  retry: very limited
  idempotency: required
  degraded: no

Recommendation Service
  timeout: 80ms
  retry: 0
  idempotency: n/a
  degraded: yes
```

この表があると、`全部同じ retry policy` や `全部同じ timeout` といった雑な設定を避けやすくなります。

読み取り系と書き込み系を分ける

RPC 設計でとくに重要なのは、読み取りと書き込みを同じ政策で扱わないことです。商品情報取得や reference data lookup は、短い cache や場合によっては hedging が効きます。一方 payment authorization や inventory reservation は、重複実行の危険があるので極めて保守的に扱うべきです。

つまり RPC は 1 つの技術でも、`read RPC` と `write RPC` は実質別物です。timeout、retry、fallback の感覚も変わります。

fallback は成功ではない

運用で誤解しやすいのが、fallback を返した瞬間に `system は正常に成功した` と見てしまうことです。実際には、fallback は `完全な結果は返せないが、より悪い失敗を避けるために縮退した` という意味です。

継続例でも、たとえば recommendation や coupon suggestion の取得が落ちたときは fallback で checkout を続けてよいかもしれません。しかし inventory や payment で同じことはできません。したがって fallback の設計では、次を分けて考える必要があります。

- user-facing に欠けてもよい情報か
- あとから補完できる情報か
- business metric 上は `成功` と数えるのか
- product team と期待値が共有されているか

fallback は transport の工夫ではなく、業務上の `ここまでなら縮退で受け入れる` という合意です。

checkout path の運用レビュー質問

最後に、この章を設計レビューへ戻しやすいように、同期 path 向けの短い質問を並べます。

1. 各依存先に個別の timeout と retry budget があるか
2. client-side queue wait を観測できるか
3. breaker が開いたときの user-facing 振る舞いは決まっているか
4. deploy 時の draining と long-lived connection の扱いは明示されているか
5. 読み取り系と書き込み系で policy を分けているか

この 5 問に答えられない同期 path は、平常時は動いても障害時に崩れやすいです。

章末まとめ

RPC を運用するとは、network 越しの call を 1 本送ることではなく、動的な接続先と queue を制御することです。service discovery、load balancing、connection pool、retry budget、breaker、telemetry が一体であり、どれか 1 つだけ整っていても十分ではありません。

継続例の checkout path で重要なのは、`Inventory Service` と `Payment Service` を同じ「依存先」として雑に扱わないことです。latency も retry 可否も failure meaning も違う以上、同期通信の設定も分けて考える必要があります。

この章の設計判断の要点

RPC を運用するとは、client library を import することではありません。service discovery、balancing、connection queue、retry budget、breaker、tracing をまとめて設計することです。local call と違い、通信コストと failure mode が動的だからです。

注文システムの継続例で言えば、`Inventory Service` と `Payment Service` は単なる helper 関数ではありません。異なる throughput、異なる失敗、異なる retry 可否を持つ独立した依存先です。したがって `同じ RPC だから同じ設定` にしてはいけません。

> **注意**
> `RPC framework が全部やってくれる` という期待は危険です。framework は transport を助けますが、retry 可否、error contract、budget 配分、degraded response の判断までは決めてくれません。

演習

1. あなたの system で最も長い synchronous call chain を書き出し、user-facing request の SLO と見比べてください。
2. `Inventory Service` と `Payment Service` に同じ retry policy を適用してはいけない理由を説明してください。
3. client-side queue wait が p99 latency の大半を占めていた場合、どこを疑うべきか整理してください。
4. unary ではなく streaming を使う価値がある場面を、注文システム以外でも 2 つ挙げてください。

この章の出口

同期通信が「呼び出し」ではなく「動的な接続先と queue を持つ運用対象」だと見えたところで、今度は同期 path から外した仕事の置き場を考えます。次章では Kafka と log のモデルを使って、その逃がし先を整理します。

Kafka、log、producer 設計

前章までで、user-facing critical path を同期 RPC で設計する感覚を作りました。しかし注文システム全体を見ると、それだけでは明らかに苦しい部分があります。メール送信、分析イベント、検索 index 更新、fraud review などは、どれも重要ですが、注文ボタンを押した user をその場で待たせる理由は薄いからです。そこで必要になるのが broker と append-only log です。この章では、`order-created` event を題材に、Kafka を「高機能な queue」ではなく、「後から複数 consumer が読める log」として整理します。

以後この章では、概念名は `OrderCreated`、topic 名や event type 名は `order-created` のように書き分けます。概念、log 上の名前、コード識別子を混ぜないためです。

この章の到達点
この章では次を押さえます。

- broker と log が必要になる理由
- command と event の違い
- Kafka の `topic`、`partition`、`offset` が何を表すか
- producer が決めるべき key、schema、ack、retry の意味
- append 成功と処理完了が違うこと

どの仕事を同期 path から外すのか

注文作成のあとにやりたい仕事を挙げると、すぐに同期 path に入りすぎる傾向があります。確認メールも送りたいし、analytics にも流したいし、検索 index も更新したいし、coupon 利用履歴も記録したい。これらを全部 `RPC` で直列に呼べば、もちろん動くことは動くでしょう。しかし checkout の latency と failure surface は急速に悪化します。

そこで最初に決めるべきなのは、「何を後でよい仕事として切り出すか」です。継続例では、少なくとも次を非同期に回します。

- 注文確認メール
- 解析イベント反映
- 検索 index 更新
- recommendation feature への入力

これらに共通しているのは、`注文が発生した` という事実をもとに、それぞれが独立して処理できることです。ここで同期 `RPC` fan-out ではなく、event log が向きます。

command と event を区別する

Kafka を導入するとき、最初に区別したいのは command と event です。

- command 誰か特定の相手に「これをやってほしい」と依頼するもの
- event すでに起きた事実を「こうなった」と記録するもの

`SendOrderConfirmationEmail` は command 的であり、`OrderCreated` は event 的です。Kafka に載せるときは、後者の見方のほうが自然です。なぜなら Kafka は、複数 consumer がそれぞれの都合で同じ record を読みたいときに強いからです。

`OrderCreated` が log に書かれていれば、Email Worker はメール送信に使い、Analytics Consumer は集計に使い、Search Indexer は検索 document 更新に使えます。1 つの command を 3 回複製して送るより、1 つの event を 3 つの consumer が読むほうが自然です。

queue ではなく log として理解する

Kafka を単なる queue として理解すると、すぐに説明しにくい現象にぶつかります。なぜ同じ message を複数の consumer group が読めるのか。なぜ古い record を replay できるのか。なぜ offset が consumer ごとに別なのか。これらは queue というより log として見ると理解しやすくなります。

log として見ると、Kafka の責務は比較的単純です。

- producer から record を append する
- append された順序を partition 単位で保持する
- consumer ごとの読了位置は consumer 側が持つ

つまり Kafka 自体は「もう処理し終えたから消す」という強い意味を持ちません。保持しているのは log であり、誰がどこまで読んだかは consumer group ごとの状態です。

```
topic: order-events
partition-0: [offset 0][offset 1][offset 2][offset 3]...
partition-1: [offset 0][offset 1][offset 2]...

email-group       -> partition-0 offset 3, partition-1 offset 1
analytics-group   -> partition-0 offset 2, partition-1 offset 2
search-group      -> partition-0 offset 3, partition-1 offset 0
```

この図から分かるように、同じ record を各 group が別の速度で追えます。これが Kafka を「複数下流への fan-out 基盤」として使いやすくしている理由です。

topic、partition、offset

Kafka を読むときの最小語彙は次の 3 つです。

- `topic` 同種の record を置く論理名
- `partition` 並列性と順序保証の単位
- `offset` partition 内の位置を表す連番

ここで特に重要なのは、順序保証は通常 partition 単位だということです。`order-events` topic 全体に 1 本の total order があるわけではありません。したがって、どの entity の順序を守りたいかに応じて key を決める必要があります。

継続例では、`order_id` を key にするのが自然です。Kafka のデフォルトパーティショナーはキーの murmur2 ハッシュを使うため、同じ注文に関する `OrderCreated`、`OrderPaid`、`OrderCancelled` は partition 数が変わらない限り常に同じ partition へ送られ、注文単位の順序を保てます。

partition key は business decision である

多くの初学者は、partition key を「適当に均等化するための文字列」と考えます。しかし実際には business decision です。何の順序を守りたいか、どの単位で並列化したいかを表します。

たとえば次のような候補があります。

- `customer_id` 顧客単位の順序を守りたいときに便利だが、ヘビーユーザが hot key になる
- `order_id` 注文単位の順序には自然だが、顧客単位の整合性は別管理になる
- `merchant_id` 店舗単位の集計には便利だが、大規模店舗に偏りやすい

どれを選んでも tradeoff があります。重要なのは、まず守りたい ordering scope を定義し、そのために key を選ぶことです。均等分散だけを目的にすると、必要な順序をあとから回復するコストのほうが大きくなります。

```
Ordering scope examples

key = order_id
  守りやすい順序: 注文単位
  向いている consumer: order lifecycle, customer notification

key = customer_id
  守りやすい順序: 顧客単位
  向いている consumer: customer timeline, fraud heuristics
```

```
key = merchant_id
  守りやすい順序: 店舗単位
  向いている consumer: merchant analytics
```

この比較をしておくと、「なぜこの key にしたのか」が後から説明しやすくなります。

event schema は何を残すべきか

`OrderCreated` event を設計するとき、迷うのは「どこまで payload に含めるか」です。最小限の ID だけを送り、consumer は追加情報を `RPC` で取りに行く方法もあります。一方、注文時点の金額や商品 snapshot を payload に全部入れてしまう方法もあります。

ここでの判断軸は次の通りです。

- event 単体で consumer が意味を解釈できるか
- あとから replay したとき、当時の状態を再現できるか
- payload が肥大化しすぎないか

継続例では、検索 indexer や analytics は「注文時点の情報」を後から再現したいことが多いので、`order_id` だけでなく `customer_id`、`line_items`、`total_amount`、`created_at` などを event に含める価値があります。逆に機密性の高い決済トークンなどは含めるべきではありません。

```go
type OrderCreatedEvent struct {
    EventID      string
    OrderID      string
    CustomerID   string
    MerchantID   string
    TotalAmount  int64
    Currency     string
    LineItems    []LineItemSnapshot
    CreatedAt    time.Time
    SchemaVersion int
}
```

このような schema は、consumer が `Order Service` へ追加 `RPC` せずに最低限の処理を進められるようにするためのものです。同期依存を減らすことは、非同期設計の大きな利点です。

producer が決めるべきこと

producer の責務は「publish を呼ぶ」だけではありません。少なくとも次を決めなければなりません。

- record key を何にするか
- schema version をどう持つか
- append 成功をどこまで待つか
- retry 時の ordering と重複をどう見るか
- timestamp を event time にするか append time にするか

この中でも特に重要なのは `append 成功をどこまで待つか` です。broker に書けたことを確認してから response を返すのか、outbox に記録できた時点でよしとするのかで同期 path の設計が変わります。本書では後者を採ります。理由は、Kafka 自体を critical path に直結させすぎないためです。

outbox row に何を持つか

outbox を入れるなら、row の設計も大切です。最低限ほしいのは次です。

- `event_id`
- `topic`
- `key`
- `payload`
- `created_at`
- `status`

これに加えて、再送や監査のために `attempt_count` や `last_error` を持つ設計もあります。重要なのは、relay が落ちたあとに何が pending か分かり、同じ row を安全に再処理できることです。

append 成功と処理完了は違う

Kafka を使い始めると、「publish できたから処理された」と思いがちです。しかし実際には、append 成功と consumer side effect 完了は別物です。

- producer にとっての成功 record が log に append された
- consumer にとっての成功 record を読んで自分の処理を完了した
- user にとっての成功 必要な副作用が見える形で反映された

たとえば `OrderCreated` を append できても、Email Worker が落ちていればメールは送られません。Analytics Consumer が lag していれば dashboard 反映は遅れます。したがって Kafka を入れると成功の定義が多層になります。ここが同期 `RPC` と大きく違う点です。

> **編集注記**
> 分散設計では「どの層の成功を見ているか」を混ぜると会話が壊れます。`publish は成功している` と `ユーザ通知まで完了している` は別の主張です。

producer retry と重複

producer も失敗します。network 切断、leader 切り替え、broker 一時停止などが起きれば、publish call は timeout するかもしれません。ここで自動 retry を入れると、多くの場合は便利ですが、「最初の append が本当に失敗だったか」を常に完全には分かりません。したがって producer 側にも重複の視点が必要です。

event に `EventID` を持たせておくのは、consumer 側 dedup や監査で役立ちます。また、outbox pattern を使うなら `outbox_id` 自体を stable identity として扱えます。大切なのは、`同じ事実の再送` を識別できることです。

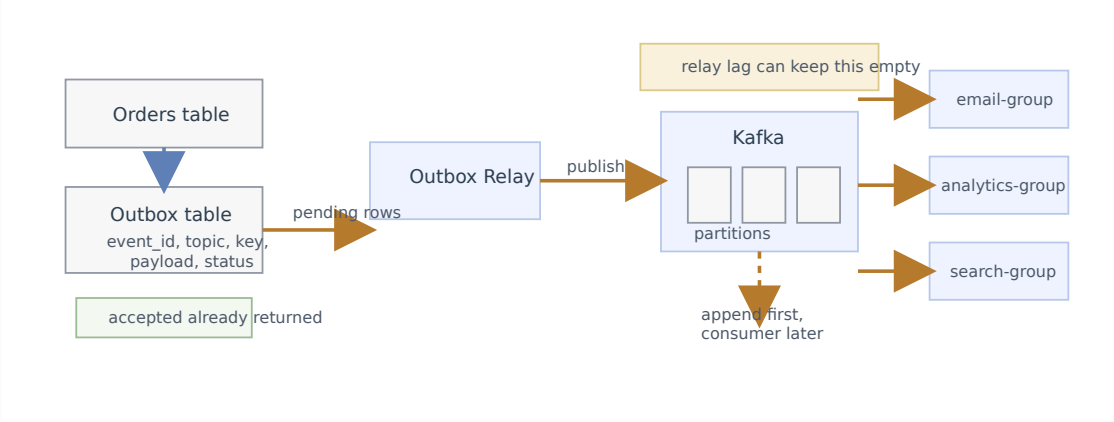
ケース 1: order row はあるが event がまだ見えない

outbox を使うときにまず理解したいのは、この状態が正常に起こり得ることです。

```
T0 order row inserted
T1 outbox row inserted
T2 client receives accepted
T3 relay is delayed or stopped
T4 Kafka does not yet contain OrderCreated
```

このとき `注文は成功しているのに analytics に見えない` という現象が起きます。これは必ずしもバグではなく、`DB state と log append のあいだに時間差がある` ことの自然な結果です。

重要なのは、この差を異常と通常で切り分けられることです。relay lag が freshness budget を超えているなら障害ですが、短時間の遅れ自体は outbox 設計の前提に含まれます。



order row と outbox row は先に確定し、その後 relay が Kafka と consumer 側の可視性を作る

event time と processing time

analytics や検索更新では、いつ起きた事実か と いつ処理されたか を分けておく必要があります。イベントが 10 分遅れで処理されることは普通にあるからです。

- event time 事実が起きた時刻
- append time broker に書かれた時刻
- processing time consumer が処理した時刻

この 3 つを混ぜると、遅延分析や replay の意味が曖昧になります。注文システムでは、売上集計を event time で行いたい場面が多いはずです。

topic をどう切るか

topic 設計も実務では重要です。よくある問いは、「`order-created`、`order-cancelled`、`order-paid` を 1 つの topic に入れるか、別 topic に分けるか」です。

1 つにまとめる利点は、同じ aggregate に関する event を 1 本の log として見やすいことです。別 topic に分ける利点は、consumer ごとに必要な event だけ読めることです。継続例では、`order-events` のような大きめの topic に event type を持たせる設計と、`order-created` などを明示的に分ける設計のどちらもあり得ます。

判断軸は次の通りです。

- 同じ consumer が複数 event type をまとめて見たいか
- ordering をどの範囲で保ちたいか
- topic 増加の運用コストをどう見るか

初学段階では、event type ごとに役割が大きく違うなら分ける、同じ aggregate の状態遷移をまとめて見たいなら 1 topic + type field もあり得る、くらいの理解で十分です。

thin event が同期依存を呼び戻す

producer 設計でありがちな失敗は、payload を削りすぎて `consumer が毎回 Order Service に問い合わせる` 形へ戻ってしまうことです。これは一見きれいですが、非同期 fan-out の価値をかなり削ります。

たとえば `order_id` だけを流し、Email Worker、Analytics Consumer、Search Indexer がすべて追加 `RPC` で注文詳細を取りに行くと、次の問題が出ます。

- Order Service が新しいボトルネックになる
- replay 時に大量の同期 call が発生する
- 注文当時の snapshot ではなく、現在値を読んでしまう
- consumer の独立性が弱くなる

thin event は悪ではありませんが、`consumer を再び同期依存へ戻していないか` を必ず確認する必要があります。

schema evolution を先に考える

Kafka では record が後から replay されるので、schema evolution は `RPC` 以上に重要です。producer を更新した瞬間だけ動けばよいのではなく、旧 consumer、新 consumer、旧 record、新 record の組み合わせを考える必要があります。

そのために最低限やっておきたいのは次です。

- 追加 field は optional にする
- consumer は未知 field を無視できるようにする
- 破壊的変更は version を分ける
- event 名自体の意味変更を避ける

`OrderCreated` が昨日までは `pending payment` を含まなかったのに、今日から含む、といった意味変更は後から効いてきます。schema は型だけでなく意味でも互換性が必要です。

retention と replay cost を見積もる

Kafka は log なので replay できる、という説明はよく出ます。しかし replay は無料ではありません。保持期間、topic size、consumer throughput、downstream の受け皿を見積もらなければ、いざ再処理したいときに現実的ではなくなります。

継続例でも、`Search Indexer` の再構築と `Analytics Consumer` の backfill では意味が違います。前者は時間がかかってもよいが、検索 freshness への影響を読まなければいけません。後者は重い再集計で warehouse を詰まらせるかもしれません。replay を設計に含めるなら、少なくとも次を決めておくべきです。

- 何日分を保持したいのか
- 最大どのくらいの速度で replay するのか
- replay 中に通常 traffic をどう守るのか
- replay 済みかどうかをどう識別するのか

`Kafka だからあとから何度でも読める` ではなく、`そのための容量と運用手順を先に持つ` と理解したほうが実務に近いです。

compaction が向く場面、向かない場面

Kafka には compaction という考え方があります。同じ key に対する古い record を整理し、最新値を取り出しやすくする発想です。これは便利ですが、すべての topic に向くわけではありません。

- 向いている例 最新状態の projection を再構築したい topic
- 向きにくい例 全履歴そのものに意味がある監査イベント

注文システムなら、`order-state-snapshots` のような `最新状態をすぐ得たい` topic には検討余地があります。一方 `order-events` のように `作成された、支払われた、取消された` の全履歴に意味がある log では、単純に compaction へ寄せると過去の因果が見えにくくなります。

重要なのは、`log を残したいのか、最新状態を引きたいのか` を topic ごとに区別することです。

producer 側にも backpressure がある

Kafka を使うと `consumer lag` ばかり目立ちますが、producer 側にも backpressure はあります。broker が遅い、ack を待つ、buffer が詰まる、といった状況では producer 自体が block したり error を返したりします。

ここで設計が問われるのは、注文受付の同期 path に Kafka publish を直接入れていないか、という点です。継続例で outbox を使うのは、producer 側 backpressure が checkout path を直撃しないようにするためでもあります。Kafka は強力ですが、それ自体が依存先である以上、同期 path に近づけるほど failure surface も増えます。

relay は producer の一部である

outbox を採ると、application code の `publish()` 呼び出しは同期 path から消えます。しかし producer の責務が消えるわけではありません。それは relay へ移るだけです。

relay 側でも少なくとも次を決める必要があります。

- pending row をどの順で読むか
- 失敗時の retry と backoff をどうするか
- `sent` への遷移をどこで確定するか
- stuck row をどう観測するか

つまり relay は単なるバッチ job ではなく、producer の後半です。ここを雑に実装すると、outbox を入れたのに event delivery の信頼性が上がらないことがあります。

Kafka は非同期 fan-out の土台

ここまでの話を注文システムへ戻すと、Kafka の役割はかなりはっきりします。Order Service が同期 path の最後で `order-created` を outbox に記録し、後段が Kafka へ流す。Email、Analytics、Search はそれぞれの consumer group で読んで処理する。1 つの事実を複数の関心事が独立に消費する構造です。

これにより、Order Service は「注文を受け付けること」に集中し、下流 consumer はそれぞれ自分の SLO で処理できます。まさに疎結合の利点です。

producer 設計の比較表

```
Producer design choices

1. key = order_id
   向いている: 注文ライフサイクルの順序を見たい consumer
   注意点: 顧客単位の偏りは別途吸収が必要

2. payload = thin event (IDs only)
   向いている: 厳密な source of truth を常に RPC で取りに行く設計
   注意点: consumer が再び同期依存を持ちやすい

3. payload = snapshot-rich event
   向いている: replay, analytics, index rebuild
   注意点: schema evolution と payload size の管理が必要
```

この比較を見ると、producer 設計は `publish call の書き方` ではなく、consumer の未来まで含んだ意思決定だと分かります。

topic 命名と ownership を先に決める

producer 設計で地味に重要なのが、topic 名を 事実 として表現し、その owner を明確にすることです。`send-email` のような command 的な名前と、`order-created` のような event 的な名前では、consumer の期待が変わります。

ここで曖昧さを残すと、topic が次第に 何でも流す共有バス になりやすくなります。そうなると schema 変更も owner も曖昧になり、結局は replay や debugging が難しくなります。

最低限決めておきたいのは次です。

- topic 名は命令ではなく事実を表しているか
- producer owner は誰か
- schema 変更レビューは誰が持つか
- retention と replay の方針は誰が決めるか

Kafka の topic は queue 名ではなく、組織的な契約名でもあります。

producer 設計レビューの問い

producer 側のレビューでは、次の 6 問があると議論しやすくなります。

1. この topic 名は事実を表しているか
2. payload だけで主要 consumer は処理できるか
3. key は ordering scope を表しているか
4. replay したとき payload の意味は保てるか
5. relay / retry による重複を `EventID` で追えるか
6. retention と owner は誰が決めるか

この問いに答えられない topic は、あとから 何のための `log` か が曖昧になりやすいです。

章末で見る producer の責務

producer は イベントを投げる人 ではなく、system の境界を定義する人です。どの field を残すか、どの key を使うか、append 成功をどの段で観測するかによって、consumer の自由度と整合性のコストが変わります。

だからこそ producer 設計は、Order Service の内部実装 detail ではなく、ドメイン設計の一部として扱うべきです。

> **注意**
> Kafka を使えば設計が楽になるわけではありません。同期 path の latency と引き換えに、lag、replay、ordering、schema evolution、consumer ownership の問題を引き受けることになります。

章末まとめ

Kafka の本質は、message をどこかへ送ることではなく、起きた事実を `log` に残し、複数の `consumer` が独立進捗で読む ことです。そのため、producer 設計は key、schema、payload、append 成功の意味まで含んだ architectural decision になります。

継続例では `order-created` を outbox 経由で Kafka へ流すことで、checkout の同期 path を短くしつつ、Email、Analytics、Search が自分の速度で追える形を作っています。ここが request/response だけでは得られない価値です。

この章の設計判断の要点

producer 設計で先に決めるべきなのは、broker の設定値より `何を事実として残すか` です。topic 名、key、payload、owner を曖昧にしたまま publish だけ先に作ると、consumer 側で同期依存や schema 破壊を呼び込みやすくなります。

注文システムで言えば、`order-created` は `Email Worker` への命令ではなく、注文が成立したという事実です。この見方を固定すると、後段 consumer ごとに速度も責務も変えられるようになります。

演習

1. `order-created` event に最低限必要な field と、含めるべきでない field を書き分けてください。
2. あなたの system で partition key を選ぶとき、何の順序を守りたいかを 1 文で定義してください。
3. `publish 成功` と `業務完了` がずれる例を 3 つ挙げてください。
4. command と event を混同すると、どのような consumer 設計の混乱が起きるか整理してください。

この章の出口

Kafka を append-only log として置く意味が見えたら、次に問うべきは `誰がその log を引き受けるのか` です。次章では consumer 側へ視点を移し、lag、rebalancing、DLQ、backpressure を通して責務の置き場所を見ます。

consumer、lag、backpressure

Kafka を導入すると、つい producer 側に目が向きます。どの topic に書くか、どの key を使うか、どの schema にするか。しかし実務で苦しみやすいのは、その先の consumer 側です。誰がどの partition を読むのか、処理に失敗したらどこまで戻すのか、lag が増えたら何を疑うのか、poison message をどう隔離するのか。非同期化で得た疎結合は、consumer 運用の責務と引き換えです。この章では `Email Worker`、`Analytics Consumer`、`Search Indexer` を使って、Kafka consumer の設計を整理します。

この章の到達点

この章では次を押さえます。

- consumer group が何を並列化しているのか
- offset commit と side effect 完了の関係
- lag を throughput 問題ではなく設計信号として読む方法
- rebalancing、poison message、DLQ、redrive の意味
- backpressure をどこでかけるべきか

3 つの consumer は同じではない

継続例では、`order-created` を少なくとも 3 つの consumer が読みます。

- `Email Worker` 確認メール送信。外部メール API に依存し、再送時の重複通知が問題になる
- `Analytics Consumer` 集計更新。多少遅れてもよいが、長期的には欠損を避けたい
- `Search Indexer` 検索 document 更新。replay や backfill が比較的しやすい

この 3 つは、同じ event を読んでいても性質が違います。したがって consumer 設計も同じではいけません。メール送信では idempotency と provider 制約が重要になり、analytics では event time と replay が重要になり、search では bulk 処理と index versioning が重要になります。

つまり Kafka を入れたからといって「下流は全部同じ worker」で片付くわけではありません。consumer ごとに failure mode と SLO が違うのです。

```
Consumer comparison

Email Worker
  重要なもの: idempotency, provider rate limit, duplicate suppression
  許容 lag: 小さめ
  replay safety: 低い

Analytics Consumer
  重要なもの: throughput, event time, backfill, aggregation correctness
  許容 lag: 中〜大
  replay safety: 高い

Search Indexer
  重要なもの: upsert, bulk update, rebuild, schema/index version
  許容 lag: 中
  replay safety: 比較的高い
```

consumer group は何を分担するのか

consumer group の役割は、「同じ論理 consumer を複数 instance で水平分割する」ことです。重要なのは、group ごとに独立した責務を持つ点です。

- `email-group` 各注文についてメール送信を 1 回やる責務
- `analytics-group` 各注文について集計を 1 回以上取り込む責務
- `search-group` 各注文について検索 document を更新する責務

同じ topic を読んでいても、group が違えば処理は独立です。ここを理解すると、「なぜ同じ record を複数 consumer が読めるのか」が自然になります。Kafka は `1 message = 1 consumer` ではなく、`1 record = group ごとに 1 回の処理機会` を与える仕組みなのです。

offset commit の意味

consumer を実装するとき、最初に出てくるのが offset commit です。ここで最も大事なのは、`commit した` と `業務処理が安全に終わった` を同一視しないことです。

典型的な流れは次の通りです。

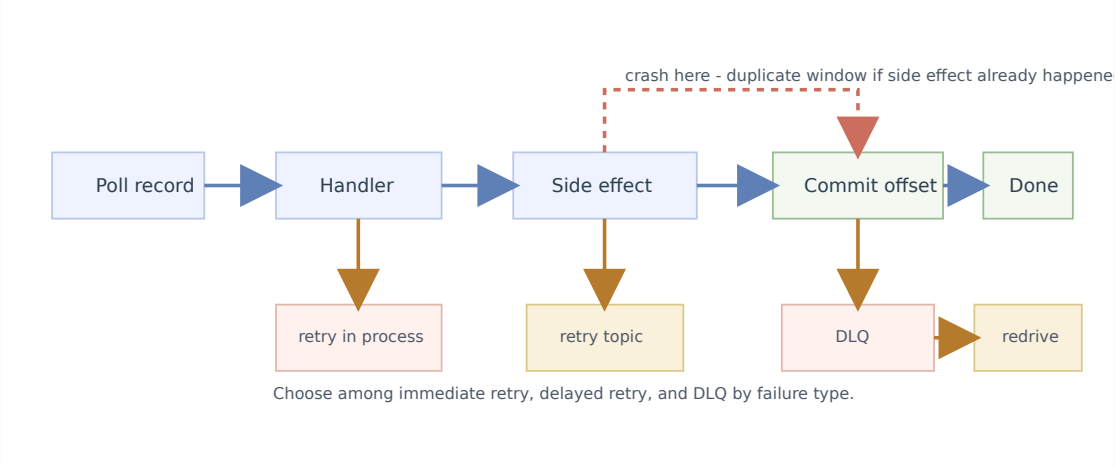
1. broker から record を poll する
2. その record に対する side effect を実行する
3. side effect が安全に完了したことを確認する
4. そのあとで offset を commit する

順序を逆にすると、commit 済みなのに side effect が失敗して record を失う危険があります。逆に commit を遅らせすぎると、重複再処理の窓が広がります。ここが at-least-once / at-most-once の話につながります。

```go
for {
    records := consumer.Poll(ctx)
    for _, rec := range records {
```

```
        event := decodeOrderCreated(rec.Value)
        if err := emailSender.Send(event); err != nil {
            markForRetry(rec, err)
            continue
        }
        if err := consumer.Commit(rec); err != nil {
            log.Warn("commit failed", "offset", rec.Offset, "err", err)
        }
    }
}
```

この擬似コードの問題点は、`Send` と `Commit` のあいだで process が落ちたら重複送信が起き得ることです。したがって `Email Worker` には idempotent な送信設計が必要になります。



consumer は side effect、offset commit、retry topic、DLQ、redrive をまとめて設計する必要がある

consumer の並列化はどこで行うのか

consumer の throughput を上げたいとき、単純に instance 数を増やしたくなります。しかし並列化の場所は 1 つではありません。

- partition 数を増やす group 全体の並列度上限が増える
- consumer instance 数を増やす partition 所有を分散できる
- 1 partition 内で handler worker を増やす 処理 throughput は上がるが、順序維持が難しくなる
- batch 化する 1 record あたりの固定コストを減らせる

ここで大切なのは、何の順序を守りたいかです。`order_id` 単位の順序が必要なら、同じ order に属する record を同時に別 worker が処理しない工夫が要ります。並列化は無料ではなく、ordering scope と引き換えです。

lag を設計信号として読む

consumer lag は「まだ読み終えていない量」を表します。多くの運用では lag が増えると「worker を増やそう」で終わりがちですが、それだけでは不十分です。lag は system がどこかで入力速度に追いつけていないサインです。

lag 増加の原因は大きく分けて次です。

- handler が遅い 外部 API、DB、重い CPU 処理

- partition が偏っている hot key が一部 partition に集中
- rebalance や deploy が多すぎる consumer が安定して働けていない
- retry / poison message 同じ record で足踏みしている

したがって lag を見たら、「平均 throughput が足りない」のか「一部 partition だけが詰まっている」のかを分けて見る必要があります。単一の平均値では分かりません。

```
lag dashboard
  email-group / partition-2    15
  analytics-group / partition-2 54000
  analytics-group / partition-7 30
  search-group / partition-2   10
```

この例なら、`analytics-group` の `partition-2` だけが明らかにおかしいことが分かります。単なる instance 不足より、hot partition や poison message を疑うべきです。

rebalancing のコスト

consumer group の便利さは、instance が増減しても partition を再配分できることです。しかし rebalance は無料ではありません。所有者が切り替わる瞬間、処理は一時停止し、in-flight record の扱いも慎重さが要ります。

rebalancing が高頻度で起きる原因としては次があります。

- autoscaling が敏感すぎる
- consumer process が不安定ですぐ落ちる
- deploy が頻繁で rolling update が長い
- session timeout や heartbeat 設定が実態に合っていない

この問題を単に Kafka の都合と見るのは危険です。rebalancing の頻発は、consumer を安定運用できていないという architectural signal です。

poison message は「壊れた 1 件」では済まない

poison message とは、特定 record だけが何度処理しても失敗する状態を指します。schema 不整合、想定外の field 値、外部依存の永続的 4xx、アプリケーション bug などが原因です。

1 件だけなら些細に見えますが、Kafka ではその 1 件が partition の進行を止めることがあります。`Email Worker` が offset 1532 で毎回落ちれば、1533 以降へ進めないかもしれません。だから poison message は queue 全体の問題になります。

対策は主に 3 つあります。

- fail fast して警告を上げる
- retry policy を分け、即時 retry と遅延 retry を区別する
- 最終的に DLQ へ隔離する

重要なのは、DLQ を「とりあえず捨てる場所」にしないことです。何を持って poison と判断したのか、あとでどう redrive するのかを決めて初めて運用になります。

DLQ と redrive

DLQ を設計するとき、少なくとも次を決める必要があります。

- 何回失敗したら隔離するか
- 元 topic / partition / offset / error をどこまで保存するか
- redrive は手動か自動か

- redrive 時に同じ consumer code を使うのか、修復専用 job を使うのか

`Search Indexer` のように replay が比較的安全な consumer では、修正後に DLQ からまとめて redrive しやすいかもしれません。一方 `Email Worker` では、古い注文確認メールを大量にまとめて再送すると user 体験が悪化することがあります。consumer ごとに policy を分ける必要があります。

> **編集注記**
> DLQ は隔離であって解決ではありません。DLQ 件数が 0 であることより、「入ったら誰が見て、どう戻すか」が明確であることのほうが重要です。

idempotent consumer の設計

consumer は at-least-once を前提に設計するほうが安全です。つまり、同じ record を複数回処理しても業務的に破綻しないようにします。

方法は複数あります。

- processed table を持つ `event_id` を記録し、2 回目以降を無視する
- upsert を使う 検索 document のように最終状態へ収束させる
- 外部 API の idempotency key を使う メール送信 provider に stable key を渡す

継続例では、`Analytics Consumer` は processed table か集計 upsert で対応しやすく、`Search Indexer` は document upsert が自然です。`Email Worker` は provider の API が許せば idempotency key を使い、なければ送信履歴を自前で持つ必要があります。

```
func (h *EmailHandler) Handle(rec Record) error {
    event := decodeOrderCreated(rec.Value)
    if h.sentLog.Exists(event.EventID) {
        return nil
    }
    if err := h.mailer.SendConfirmation(event.OrderID, event.CustomerID); err != nil
{
        return err
    }
    return h.sentLog.MarkSent(event.EventID)
}
```

この処理も、`SendConfirmation` は成功したが `MarkSent` が失敗した、というケースを考える必要があります。idempotency は小さな boolean ではなく、境界ごとの設計です。

ケース 1: Email Worker で二重送信が起きた

もっとも起きやすい事故の一つはこれです。メール provider への call は成功したが、そのあと process が落ち、offset commit も送信履歴更新もされなかった。その結果、再起動後に同じ record をもう一度処理し、ユーザへ 2 通送ってしまう、という流れです。

```
offset 1532 polled
  -> provider send ok
  -> process crash
restart
offset 1532 reprocessed
  -> duplicate mail
```

この事故から分かるのは、`commit 後に side effect` が危険なだけではないということです。`side effect 後に commit` でも idempotency がなければ重複は起きます。だから Email Worker では `event_id` ベースの送信履歴か provider 側 idempotency が要ります。

ケース 2: Analytics lag と hot partition

analytics-group の lag だけが急増しているのに、instance 数を増やしても改善しないことがあります。典型的には `merchant_id` に偏りがあり、大規模店舗の event だけが 1 partition へ集中しているケースです。

このとき重要なのは、平均 lag ではなく partition ごとの差です。特定 partition だけが高いなら、単純な scale out より key 設計や batch 処理の見直しが効きます。consumer lag は運用数字ではなく、data model の問題を教えてくれることがあります。

ケース 3: Search Indexer の作り直し

検索 index の mapping 変更や bug 修正で、Search Indexer を全件作り直したくなることがあります。こういうとき replay が効く consumer 設計は強いです。

ただし実務では、`live traffic を処理しながら replay するのか、別 index を作って切り替えるのか、replay 中の lag alert をどう扱うか` を決める必要があります。単に Kafka に record が残っているだけでは足りません。rebuild 運用も consumer design の一部です。

backpressure はどこでかけるか

consumer の世界でも backpressure は重要です。broker からいくらでも取り込んでしまうと、consumer process 内で queue が伸び、メモリと latency が悪化します。したがって「いま処理できる分だけ読む」という設計が必要です。

かける場所としては次が考えられます。

- poll batch size を絞る
- in-flight handler 数を制限する
- downstream `RPC` 呼び出しの concurrency を制限する
- lag が閾値を超えたときに一部機能を degraded にする

たとえば `Search Indexer` は batch update が効きやすいので、大きめ batch と bounded worker pool が有効かもしれません。一方 `Email Worker` は外部 provider rate limit が厳しいなら、poll 量より送信 concurrency を制御したほうが効果的です。

consumer observability

producer と違い、consumer の observability では「どこで止まっているか」を細かく見る必要があります。最低限ほしい指標は次です。

- lag per group / partition
- records consumed per second
- processing latency
- retry count
- DLQ count
- rebalance count
- handler error taxonomy

さらに、end-to-end で `order-created` から `email-sent` まで何分かかっているかを見ると、business impact を把握しやすくなります。Kafka の監視だけ green でも、実際には user 通知が 30 分遅れていることはあり得ます。

retry topic と delay queue

poison ではないが、その場ですぐ再実行するべきでもない失敗があります。たとえば一時的なメール provider rate limit や短時間の検索 cluster 過負荷です。この場合、即時 retry で同じ consumer loop を詰まらせるより、遅延 retry 用の topic や queue に逃がす構成が有効なことがあります。

考え方は単純です。

- すぐ retry したい短い障害 in-memory retry や small backoff
- 数十秒〜数分待ちたい障害 retry topic / delayed queue
- 永続的に失敗する障害 DLQ

この分岐を持つだけで、正常 traffic と障害 traffic を分けやすくなります。

stuck partition を見分ける

lag が大きいだけでは、`全体的に遅い` のか `1 件で止まっている` のか分かりません。stuck partition を見分けるには、offset の進み方と error log を一緒に見る必要があります。

典型的には次の見え方になります。

- lag は増えているが offset は少しずつ進む throughput 不足や downstream 遅延を疑う
- lag は増えており、特定 offset で止まっている poison message や handler bug を疑う
- lag は上下に揺れ、rebalance count も増えている process instability や autoscaling を疑う

この切り分けを持っておくと、`worker を増やす` 以外の対処が見えやすくなります。

commit batch と重複窓

consumer throughput を上げるために、offset commit を batch 化することがあります。これはよくある最適化ですが、同時に `どこまで再処理され得るか` の窓を広げます。

たとえば 100 件まとめて commit する設計では、99 件目まで side effect が終わったあとに process が落ちると、かなり広い範囲が再処理対象になります。`Analytics Consumer` なら許容できるかもしれませんが、`Email Worker` では危険です。

つまり batch commit は単なる性能 tuning ではなく、`重複窓をどこまで許容するか` という設計判断でもあります。

handler 内の下流 `RPC` をどう守るか

consumer の実処理がさらに下流 `RPC` を呼ぶ場合、非同期設計だから安全だとは言えません。`Search Indexer` が検索 cluster を、`Email Worker` が provider API を呼ぶなら、そこにも timeout、retry、concurrency limit が要ります。

ここで重要なのは、consumer が lag を抱えているときほど無制限 retry をしたくなる誘惑が強いことです。しかしそれをやると、`consumer lag` が `downstream RPC の障害増幅器` になります。

したがって worker 側でも次を分けて考える必要があります。

- その場で少し retry するのか
- retry topic に逃がすのか
- DLQ に送るのか
- いったん consumer 自体を pause して downstream を守るのか

非同期 worker は同期 path の外にあるだけで、依存先保護の責務までは消えません。

pause/resume を運用に組み込む

Kafka consumer には、状況によって特定 partition や group の取り込みを一時的に弱めたい場面があります。たとえば replay 中だけ通常 traffic を優先したい、外部 provider 障害の間は Email Worker を抑えたい、といった場面です。

このとき `process を kill する` だけだと、rebalance が増えたり、復旧後の burst が大きくなったりします。そこで pause/resume や bounded intake を運用の選択肢として持っておくと、system 全体を壊しにくくなります。

重要なのは、pause する条件を事前に決めておくことです。lag、error rate、provider quota、warehouse load のどれを見て止めるのかが曖昧だと、結局は現場の即興になります。

ケース 4: replay と通常 traffic がぶつかる

Search Indexer を作り直すために replay を始めたところ、通常の注文更新まで遅くなったとします。これは replay 自体が悪いのではなく、`通常 traffic と backfill traffic を別物として扱っていない` ことが問題です。

対策としては次があります。

- replay rate を制限する
- replay を別 consumer group や別 index へ逃がす
- 通常 traffic 用の worker 枠を別に確保する
- freshness alert を replay 中だけ別評価にする

replay は便利ですが、本番 traffic と同じ資源を食う以上、通常系と分離して考えるほうが安全です。

ケース 5: Email provider が rate limit した日

Email Worker では、Kafka 自体は健康でも外部 provider の quota に当たって処理が詰まることがあります。このときにありがちな誤りは、lag を見て worker 数を増やすことです。実際には provider 側制限にさらに強く当たり、失敗率だけが上がります。

このケースで見るべきなのは次です。

- provider への 429 / quota error rate
- worker 側 retry が即時になっていないか
- 送信 concurrency が quota に対して高すぎないか
- retry topic へ逃がせているか

非同期 worker の遅さは、いつも Kafka の throughput 問題ではありません。外部依存の契約がボトルネックになっていることも多いです。

consumer ごとの redrive 方針を分ける

同じ `OrderCreated` を読む consumer でも、redrive のやり方は同じではありません。簡単な表にすると違いが見えます。

```
Redrive policy sketch

Email Worker
  目的: 通知欠損の補完
  注意: 古い通知を大量送信しない

Analytics Consumer
```

```
  目的: 欠損集計の回復
  注意: warehouse への負荷を制御する

Search Indexer
  目的: projection の再構築
  注意: live traffic と rebuild を分ける
```

この表があるだけで、`DLQ からまとめて戻せばよい` という雑な発想を避けやすくなります。

lag を見たときの 5 分トリアージ

運用では、深い分析の前に `最初の 5 分で何を見るか` を固定しておくと強いです。consumer lag なら次の順が効きます。

1. どの group / partition が高いかを見る
2. offset が進んでいるか止まっているかを見る
3. rebalance count と process restart を見る
4. downstream `RPC` / DB / provider error を見る
5. すぐ止血が必要なら pause、retry topic、DLQ を選ぶ

この順序を持つと、`とりあえず台数を増やす` に流れにくくなります。

章末で見る consumer の責務

consumer は `topic を読む人` ではありません。offset、side effect、lag、retry、DLQ、redrive を束ねて、自分の責務を時間的に引き受ける component です。

producer が `どんな事実を残すか` を決めるなら、consumer は `その事実をどの速度で、どの失敗許容で、どこまで正確に反映するか` を決めます。非同期設計の本体は、ここにあります。

典型的には次のシグナルがあります。

- 同じ offset で同じ error が繰り返される
- CPU や network は低いのに lag だけ増える
- 他 partition は進んでいる

この状態では、worker を増やしても解決しません。record 内容、schema、外部依存の入力条件を見るべきです。

batch handler と partition-local state

consumer の throughput を上げるために batch handler を使うことがあります。analytics や search index 更新では、1 record ごとより 100 件まとめて処理したほうが効率的です。ただし batch 化すると、どこまで成功してどこで失敗したかが曖昧になりやすくなります。

また、一部の consumer は partition-local state を持ちたくなります。たとえば `merchant_id` ごとの集計 cache を partition 単位で持つと速いことがあります。しかし rebalance が起きると state の所有者が変わるため、warmup や flush の設計が要ります。

つまり batch と local state は性能を上げますが、rebalance と failure recovery の複雑さも上げます。性能最適化は consumer model の一部です。

redrive の実務手順

DLQ や retry topic を設けても、redrive 手順が曖昧だと本番では使えません。実務で最低限必要なのは次の順序です。

1. 隔離理由を確認する

2. 同じ失敗が live traffic でも続いていないか確認する
3. consumer code / schema /外部依存を修正する
4. 小さなサンプルで再投入し、重複や副作用を確認する
5. 問題なければ本体を段階的に redrive する

ここでいきなり大量 redrive すると、古い障害 traffic が live traffic を押し潰すことがあります。redrive も backpressure を持つべきです。

consumer 設計レビューの問い

consumer 側レビューでは、少なくとも次を確認すると抜け漏れが減ります。

1. side effect 完了と offset commit の順序は明示されているか
2. replay-safe か、そうでないかが決まっているか
3. lag がどの freshness budget を超えると障害か
4. retry topic、DLQ、pause の使い分けがあるか
5. redrive の owner と手順が runbook 化されているか

consumer は application loop に見えますが、実際には運用設計そのものです。

章末まとめ

consumer 設計の本質は、「何を 1 回に見せたいか」「何を replay 可能にしたいか」「どの lag まで耐えられるか」を consumer ごとに定義することです。同じ topic を読んでいても、Email、Analytics、Search は全く同じではありません。

非同期基盤は一見 generic に見えますが、実際の責務は極めて domain-specific です。だから consumer 章では常に 誰に何が起きるか を考え続ける必要があります。

この章の設計判断の要点

非同期設計の中心は producer ではなく consumer です。なぜなら、実際の side effect は consumer 側で起きるからです。lag、rebalancing、poison message、DLQ、idempotency を扱えるようになって初めて、Kafka は本番で役に立ちます。

継続例に戻ると、`Email Worker`、`Analytics Consumer`、`Search Indexer` は同じ `order-created` を読んでいても、処理特性も失敗時挙動も違います。ここを全部同じテンプレートで扱わず、consumer ごとに責務を言語化するのが重要です。

> **注意**
> Kafka 導入後の典型的な失敗は、「publish できたから終わり」と思って consumer 側を後回しにすることです。実務では、下流 consumer の設計不足がもっとも長く痛みます。

演習

1. `Email Worker`、`Analytics Consumer`、`Search Indexer` で、それぞれ許容できる lag の大きさがなぜ違うか説明してください。
2. consumer lag が急増したとき、instance 数不足と hot partition をどう見分けるか整理してください。
3. あなたの system で poison message が 1 件出たとき、どの partition / job / user impact が止まるかを書き出してください。
4. `commit 前に side effect` と `side effect 前に commit` のどちらがどの semantics に寄るか、具体例で比較してください。

5. ある consumer を replay-safe にするために、payload と handler をどう変えるべきか設計してください。
6. あなたの system の DLQ について、redrive の owner と段階的手順を書き出してください。

この章の出口

consumer を side effect と lag を管理する本体として見ると、次に必要なのは hop をまたいだ見方です。次章では同期 `RPC` と consumer の振る舞いをつなぎ、end-to-end consistency を整理します。

delivery semantics、ordering、整合性

前章までで、同期 `RPC` と Kafka consumer の設計を別々に見ました。しかし現実の system では、1 回の業務操作がその両方をまたぎます。注文 API が inventory と payment を同期的に呼び、DB に注文を書き、あとで Kafka に event を流し、consumer がメールや検索更新を行う。ここで問われるのは、「結局この注文は何回作られたことになるのか」「どの順序が守られるのか」「途中で落ちたらどこから回復するのか」です。この章では delivery semantics と consistency を end-to-end の視点で整理します。

この章の到達点

この章では次を押さえます。

- at-most-once / at-least-once / exactly-once を end-to-end でどう読むか
- ordering scope をどこで定義するか
- dedup、processed table、upsert をどう使い分けるか
- outbox pattern と saga の役割
- replay / backfill を前提にした設計

semantics は transport のラベルではない

at-most-once、at-least-once、exactly-once は有名な言葉ですが、transport の feature label として読むと危険です。本当に知りたいのは、「user や業務から見て、その操作が何回起きたように見えるか」です。

たとえば `Order Service -> Payment Service` の `Authorize` call は idempotency key なしだと、timeout 後 retry により二重オーソリが起きるかもしれません。Kafka で `order-created` を publish できても、consumer が 2 回処理すればメールは 2 通送られるかもしれません。つまり end-to-end semantics は、各 hop の組み合わせで決まります。

at-most-once の具体像

at-most-once は「失われる可能性はあるが、重複はしない」方向です。たとえば次のような設計です。

- timeout 後に retry しない `RPC`
- consumer が side effect 前に offset を commit する
- best-effort logging や metrics event

この設計が悪いとは限りません。たとえば low-value analytics や debug event なら、少し落としてもよいことがあります。しかし billing、inventory、user notification のような重要機能では危険です。重要なのは、「落ちてもよいもの」にだけ意識的に適用することです。

at-least-once の具体像

実務で最もよく見るのは at-least-once です。失わないことを優先し、重複は `application` で吸収するという立場です。

典型例は次です。

- timeout 後に retry する `RPC`
- side effect 完了後に offset を commit する consumer
- outbox relay が publish 成功まで同じ row を再送し得る構成

この世界では、重複は異常系ではなく通常系です。したがって「重複が起きるかもしれない」ではなく、「いつも起きる前提で吸収する」と考えるほうが安全です。

exactly-once をどう読むか

exactly-once は魅力的ですが、境界を限定せずに使うと会話が壊れます。多くの場合 exactly-once は、「ある storage / transaction の中で二重適用を見せない」ことを指します。system 全体のあらゆる外部 side effect まで魔法のように 1 回にしてくれるわけではありません。

たとえば Kafka transaction が有効でも、外部メール API や外部 payment gateway への call まで自動で exactly-once にはなりません。そこで重要なのは、「どの境界で何を 1 回に見せたいのか」を具体化することです。

> **編集注記**
> exactly-once を使いたくなったら、必ず「どの境界の中で」「何に対して」「どの観測者から見て」成立する主張なのかを書き下してください。そこが曖昧な exactly-once は設計文書として役に立ちません。

```
Semantics comparison

at-most-once
  失う可能性: ある
  重複: 起きにくい
  向いている: low-value telemetry

at-least-once
  失う可能性: 減る
  重複: 起き得る
  向いている: 多くの業務処理

exactly-once (bounded)
  失う可能性: 境界内ではなくなる
  重複: 境界内では起きない
  向いている: 特定 storage / transaction の中
```

ordering scope を決める

順序は強い制約です。だからこそ scope を絞る必要があります。システム全体で total order を守ろうとすると throughput も可用性も厳しくなります。多くの場合、本当に必要なのは次のような局所順序です。

- 同じ `order_id` に関する更新順序
- 同じ `customer_id` に関する会員状態更新順序
- 同じ `merchant_id` に関する売上締め処理順序

継続例では、注文ライフサイクルの遷移を見る consumer にとっては `order_id` 単位の順序が重要です。一方 analytics 集計では厳密な total order は不要で、event time に基づいて後で吸収できることも多いでしょう。用途によって順序要件は違います。

reorder は自然に起きる

順序が崩れる原因はたくさんあります。

- `RPC` retry が遅れて届く
- producer が別 partition へ送る
- consumer が内部で parallel 処理する
- rebalance で一時停止し、別 partition だけ先に進む
- outbox relay の再送で append 時刻がずれる

したがって「順序が大事」と言うだけでは足りません。受け手側で古い更新を見分ける仕組みが必要です。version number、state machine validation、monotonic sequence、event time watermark などがそのために使われます。

dedup のやり方

重複吸収の方法は、対象によって変わります。代表的なのは次の 3 つです。

- processed table `event_id` や `idempotency_key` を保存し、2 回目以降を無視する
- upsert 同じキーに対して最終状態を書き込む
- versioned update 新しい version だけを受け付け、古い更新を拒否する

`Email Worker` なら processed table や送信履歴が効きやすく、`Search Indexer` なら upsert が自然です。注文 status の更新 consumer では versioned update が向いていることがあります。重要なのは、`consumer だから processed table` と決め打ちしないことです。

```
func (p *ProcessedStore) Seen(eventID string) bool

func HandleOrderCreated(rec Record) error {
    event := decode(rec.Value)
    if processed.Seen(event.EventID) {
        return nil
    }
    if err := projector.UpsertOrder(event); err != nil {
        return err
    }
    return processed.MarkSeen(event.EventID)
}
```

この形でも、`UpsertOrder` 成功後に `MarkSeen` が失敗すれば再処理は起きます。だからこそ、projector 自体も idempotent であるほうが安全です。

idempotency key の寿命を決める

idempotency key は導入しただけでは不十分で、どのくらい保持するかを決める必要があります。短すぎると遅延 retry を吸収できず、長すぎると storage と lookup cost が増えます。

継続例では、payment authorization の key は 同じ注文に対する重複オーソリを防ぐ ことが目的です。したがって保持期間は、network timeout 数秒ではなく、user の再送、mobile client の再試行、operator の手動再実行まで含めて考えるべきです。

ここでの問いは次です。

- 重複が最も遅れて再到達し得るのはいつか
- provider 側の idempotency window と整合しているか
- key と request body の対応を検証するか
- 期限切れ後の再送をどう扱うか

idempotency は 重複しない魔法の文字列 ではなく、寿命を持つ契約です。

end-to-end invariant を先に書く

整合性の議論で役立つのは、技術選択の前に この業務操作で破ってはいけない条件 を短く書くことです。継続例なら、たとえば次のように書けます。

- 1 つの注文 intent から確定注文は高々 1 つ
- payment authorization は同じ intent で重複課金しない
- `accepted` を返した注文は、あとから追跡可能である
- 通知や検索反映は遅れてもよいが、欠損は検知できる

この invariant があると、どこで `idempotency` を持つべきか、どこで `replay` を許すか、どこで `manual recovery` が必要か が議論しやすくなります。整合性は抽象的な美しさではなく、破ってはいけない条件の集合です。

outbox pattern

`Order Service` が同期 `RPC` で inventory と payment を済ませたあと、DB へ注文 row を書き、Kafka に `order-created` を publish したいとします。ここで最も危険なのは、`DB 書き込み成功 / publish 失敗` や `publish 成功 / DB 書き込み失敗` のズレです。

この問題に対して強いのが outbox pattern です。つまり、本来の state 変更と「あとで publish すべき event」を同じ DB transaction に書く方法です。

```
DB transaction
  1. insert into orders(...)
  2. insert into outbox(event_id, topic, payload, status='pending')
commit

Outbox Relay
  3. read pending rows
  4. publish to Kafka
  5. mark outbox row as sent
```

この構成なら、「注文 row はあるが event が永遠に失われる」をかなり減らせます。relay が落ちても pending row は残るからです。

outbox が解決しないこと

outbox は強力ですが万能ではありません。解決するのは、主に `DB state` と `publish request` のずれです。consumer 側の side effect や外部 API 呼び出しまで自動で整合させてくれるわけではありません。

また relay は再送し得るので、Kafka 側や consumer 側では依然として重複を扱う必要があります。つまり outbox は exactly-once の魔法ではなく、「もっとも危険な境界の一つを整理する」道具です。

```
Dual write vs outbox

Dual write
  1. write DB
```

```
  2. publish
  問題: 途中失敗でズレやすい

Outbox
  1. write DB + outbox in one tx
  2. relay later
  利点: publish request の欠落を減らせる
  注意: relay/consumer 側の重複処理は残る
```

saga と補償

注文作成では、inventory reservation と payment authorization を別 service が持っている以上、単一 DB transaction のようには扱えません。そこで出てくるのが saga 的な考え方です。つまり、各 service がローカルに変更を行い、途中失敗時には補償 action を流すという設計です。

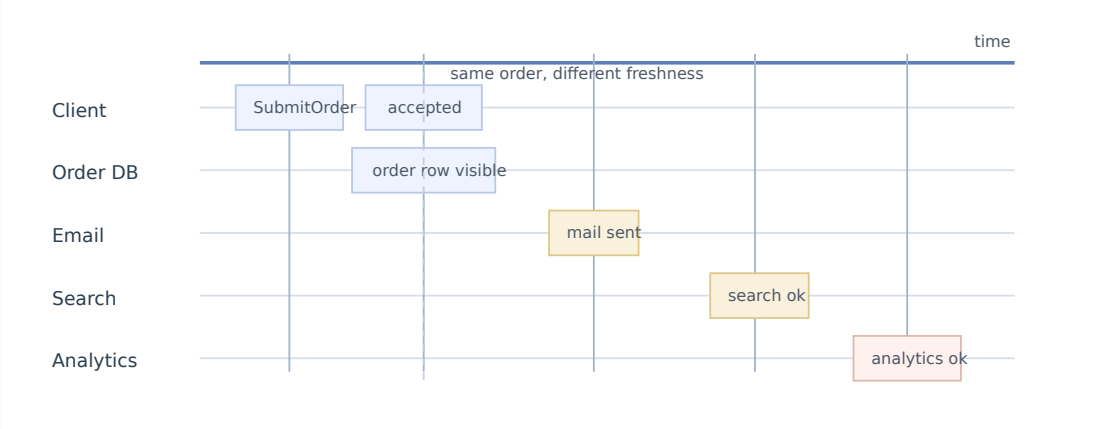
継続例では、たとえば次のような補償が考えられます。

- payment authorization に成功後、order DB 書き込みが失敗 `CancelAuthorization` をベストエフォートで呼ぶ
- inventory reservation に成功後、payment が拒否 `ReleaseReservation` を呼ぶ
- order created 後、fraud review で拒否 `OrderCancelled` event を新たに流す

ここで大切なのは、補償は rollback と同じではないことです。外部 gateway の取消は完全に元へ戻す保証ではないかもしれませんし、遅延や手動介入が必要なこともあります。saga は「完全な巻き戻し」ではなく、「業務的に許容できる状態へ戻す」仕組みです。

具体例: 注文の end-to-end timeline

継続例を end-to-end で書くと、次のような timeline になります。



同じ注文でも、client、DB、mail、search、analytics では見える時刻がずれる

この timeline では、client が成功を観測するのは `T6` です。しかし全ての副作用が終わるのは `T10` 以降です。ここを切り分けて考えることが、サービス間通信の設計そのものです。

ケース: accepted だがメールはまだ送られていない

実務でよく起きる混乱は、この状態を 失敗 と呼ぶ人と 正常 と呼ぶ人が混在することです。

```
T6 client sees accepted
T7 outbox relay is healthy
```

```
T8 email-group lag grows for 3 minutes
T9 order detail is visible, mail is not yet sent
```

本書の継続例では、これは 注文整合性の失敗 ではなく 通知 `freshness` の遅れ として扱います。もちろん遅れが budget を超えれば障害ですが、注文成立そのものとは切り分けます。

このように 何が壊れたのか を分類できると、システム全体を止めるべき障害と、degraded で耐えるべき障害を区別しやすくなります。

replay と backfill

Kafka を使う大きな利点の一つは replay です。新しい consumer を追加したい、bug 修正後に再集計したい、検索 index を作り直したい、といったときに、過去 record を読み直せます。

しかし replay を有効にするには、最初からそれを意識した schema と consumer 設計が必要です。

- event に十分な snapshot を持たせる
- consumer を idempotent にする
- external side effect を無闇に再実行しない
- event time と processing time を区別する

`Search Indexer` は replay と相性がよい consumer ですが、`Email Worker` は相性が悪いことがあります。再読可能である と 再実行可能である は違うのです。

read-your-own-writes と projection

非同期 projection を入れると、「注文を作った直後に一覧で見えない」問題が出てきます。これは read-your-own-writes の要求と projection lag の衝突です。

対策はいくつかあります。

- user-facing read は同期 DB を見に行く
- projection が追いつくまで pending 表示を出す
- 特定 user だけ write-side read model を優先する

ここで大事なのは、projection lag を隠そうとして全てを同期化しないことです。user に何を見せれば自然かを product と詰めるほうが、本質的です。

freshness budget を product requirement にする

projection lag を議論するとき、`eventual consistency` という抽象語だけでは弱すぎます。必要なのは、どの read model が何秒以内に追いつくべきかという freshness budget です。

たとえば継続例では次のように分けられます。

- 注文詳細画面 ほぼ即時、もしくは write-side read で補う
- 注文確認メール 数分以内なら許容
- analytics dashboard 10 分遅延でも業務上許容
- search index 数十秒〜数分の遅れを許容

この予算が明確だと、lag alert、replay 優先度、degraded mode の会話が具体的になります。整合性は storage の性質だけでなく、product の期待値でも決まります。

cancellation event をどう扱うか

注文システムでは `OrderCreated` だけで終わりません。後から `OrderCancelled`、`PaymentFailed`、`InventoryReleased` が流れるかもしれません。このとき consumer は「最新

event だけ見ればよい」のか、「履歴全体を state machine として見るべきか」を決める必要があります。

検索 indexer なら最新状態へ upsert すればよいかもしれませんが、analytics では `created` と `cancelled` の両方を別イベントとして数えたいことがあります。つまり同じ event stream でも consumer によって意味の切り取り方が違います。

inquiry path を consistency の一部として見る

分散システムでは、`最初の request で答え切れない` 状態がどうしても出ます。payment timeout、relay 遅延、consumer lag などです。このとき重要なのは、結果確認用の inquiry path を整合性設計の一部として扱うことです。

たとえば注文システムでは、次のような問い合わせがあり得ます。

- `client_request_id` から注文成立有無を確認する
- `order_id` から現在 state を確認する
- `event_id` から consumer 側反映状況を運用者が追える

この経路があると、`unknown なら再送` ではなく `unknown なら確認` という運用が可能になります。整合性は write path だけでなく inquiry path でも支えられます。

projection rebuild と snapshot

長く運用していると、projection を丸ごと作り直したくなることがあります。検索 index の mapping を変えた、analytics の定義を変えた、新しい read model を追加した、などです。このとき replay は強い武器ですが、log が長大になるほど時間もかかります。

そこで実務では snapshot や checkpoint を併用することがあります。たとえばある日付時点の集計 state を保存し、その後ろだけ replay する構成です。ただし snapshot を導入すると、snapshot 自体の整合性と version 管理という新しい課題も入ります。最初から必要とは限りませんが、長期運用では知っておく価値があります。

saga state をどこに持つか

補償を伴う saga を実装するとき、`いまどの段階まで進んでいるか` をどこで持つかが問題になります。in-memory だけでは再起動で失われるので、永続化された state machine が必要になることがあります。

注文システムなら、`pending_inventory`、`pending_payment`、`accepted`、`cancel_requested` のような状態を order row や workflow table に持つ設計があり得ます。重要なのは、補償もまた明示的な状態遷移だと理解することです。

manual recovery path を先に決める

整合性の設計では、自動回復だけでなく手動回復の道も決めておくべきです。実務では、provider 障害、schema 破壊、長時間 lag のように、完全自動では片付かない場面が必ず出ます。

継続例なら次のような手動回復が考えられます。

- payment status を provider 側で照会して注文状態を訂正する
- DLQ の `OrderCreated` を修復後に redrive する
- `OrderCancelled` を手動投入して補償フローを進める
- 検索 index を再構築して projection を追いつかせる

ここを設計に含める理由は、自動化できないから設計対象外 ではないからです。むしろ人手介入が必要な境界こそ、事前に責務と証跡を決めておく価値があります。

unknown state を短く保つ

整合性の実務で厄介なのは、成功か失敗かまだ断言できない 状態です。payment timeout 直後や response loss 直後は、まさにこれに当たります。

この状態を完全になくすのは難しくても、短く保つ設計はできます。

- idempotency key で重複再送を吸収する
- inquiry path で後追い確認できるようにする
- provider status check を runbook 化する
- `accepted / rejected / unknown` の意味を product と共有する

unknown state を設計に入れないと、現場では結局 多分失敗だろう で再送され、重複実行が増えます。

状態機械として見る

注文の整合性を議論するとき、`created`、`paid`、`cancelled` を単なる event 名として見るだけでは足りません。consumer や UI は、最終的に何らかの state machine を見ています。

```
pending_submission
  -> reserved
  -> authorized
  -> accepted
  -> cancelled
  -> refunded
```

この状態機械を持つと、`OrderCancelled` が `OrderCreated` より先に見えたらどうするか、`PaymentFailed` が `accepted` 後に来たらどう扱うか、といった議論がしやすくなります。整合性の議論は、event 名より state transition で行うと整理しやすいです。

補償が効きにくい操作

saga を入れると何でも元に戻せそうに見えますが、実際には補償が難しい操作があります。メール送信、外部通知、物理配送、外部パートナー連携などです。

こうした操作では、完全 `rollback` を期待するより、後続の訂正イベント や 運用フロー を前提にしたほうが現実的です。だからこそ side effect の順序と user-facing meaning を慎重に決める必要があります。

consistency budget を言葉にする

最後に重要なのは、整合性もまた予算の問題だと捉えることです。完全同期、完全順序、完全 1 回実行を全部同時に求めると、可用性と複雑性のコストが急激に上がります。

したがって設計では、少なくとも次を言葉にする必要があります。

- どの操作だけは二重実行が許されないか
- どの read model は遅れてもよいか
- どの補償は自動で、どこから人手か
- どの境界で exactly-once 風の性質を求めるか

この会話ができると、整合性が大事 という抽象論から一歩出て、実際の system 制約へ落とし込めます。

consistency レビューの問い

章の最後に、整合性レビュー向けの短い問いを置いておきます。

1. この操作で破ってはいけない invariant は何か
2. unknown state が起きたとき、誰がどう確認するか
3. replay-safe な consumer と unsafe な consumer はどれか
4. manual recovery が必要な境界はどこか
5. freshness budget を超えたとき、どの user impact が出るか

整合性の議論は抽象化しやすいので、このくらい具体的な問いへ落とすと設計レビューが進みやすくなります。

章末まとめ

consistency は、Kafka や database の専門用語だけではありません。注文という業務操作が `何回起きたことになるか`、`何がいつ user に見えるか` を決める設計です。outbox、dedup、replay、saga はそのための道具です。

大切なのは `どこか一箇所で exactly-once っぽい` ことではなく、hop をまたいだときにどの性質が残るかを把握することです。

consistency を product と共有する

backend だけで整合性を議論すると、「eventual consistency だから」で片付けがちです。しかし product と共有すべきなのは、いつ何が user に見えるかです。

たとえば注文直後に検索結果へ反映されなくてもよいのか。confirmation email が 5 分遅れるのは許容か。analytics dashboard が 10 分遅れても問題ないか。こうした要件が分かって初めて、どこで synchronous にし、どこで asynchronous にし、何を replay 可能にするかが決まります。

> **注意**
> consistency は storage の専門用語であるだけでなく、user-visible state の約束でもあります。`eventual consistency` と書くだけでは設計になりません。

この章の設計判断の要点

delivery semantics と consistency は、Kafka の章だけで完結する話ではありません。`RPC` retry、idempotency key、outbox、consumer dedup、replay が全部つながって初めて、end-to-end の性質が決まります。

注文システムでは、「注文を 1 回だけ作り、必要な副作用を重複許容で進める」ことが実際の設計目標です。そのために critical path では response の意味を固定し、境界では outbox を置き、consumer 側では idempotency を持たせます。

> **編集注記**
> 分散システムで最も避けたいのは、「どこかが exactly-once らしいから全体も安全だろう」という推論です。全体の性質は hop の合成で決まります。

演習

1. あなたの system で at-least-once を許容できる処理と、できない処理を分類してください。

2. `order-created` を replay したとき、`Email Worker` と `Search Indexer` のどちらが危険か、その理由を書いてください。
3. outbox pattern が解決するズレと、解決しないズレを 2 つずつ挙げてください。
4. `order_id` 単位の順序が崩れたとき、consumer 側でどう検出・吸収するかを設計してください。
5. あなたの system の中で、state machine として定義したほうがよいイベント列を 1 つ選んでください。
6. 補償が効きにくい side effect を 3 つ挙げ、それぞれ rollback 以外の回復策を考えてください。

この章の出口

end-to-end consistency の骨格が見えたら、最後に必要なのは障害時の判断順です。最終章では observability、ownership、使い分け、回復手順を playbook としてまとめます。

障害対応と設計 playbook

ここまでで、同期 `RPC` と Kafka をどう設計するかを見てきました。しかし本番では、設計は常に障害の形で試されます。payment timeout が続く、analytics lag が急増する、schema 変更で consumer が落ちる、rebalancing が頻発する、トレースは green なのに user は確認メールが届かないと報告する。こうしたときに必要なのは、個別技術の知識より「何をどの順で疑うか」の playbook です。この章では、注文システムを例に、障害対応と日常的な設計レビューの観点を整理します。

この章の到達点

この章では次を押さえます。

- partial failure と cascading failure をどう切り分けるか
- `RPC` と Kafka の使い分けを判断表に落とす方法
- 典型的なインシデントにどう対処するか
- observability、ownership、schema governance をどう設計するか
- architecture review の観点をどう揃えるか

partial failure を通常系として扱う

分散システムでは、全部が一斉に壊れるより、一部だけが遅い・落ちるほうが普通です。`Payment Service` の一部 instance だけが 5xx を返す、`analytics-group` の特定 partition だけ lag が増える、ある consumer だけが新 schema を読めない、といった壊れ方です。

ここで重要なのは、`1 つでも成功しているから大丈夫` と見ないことです。user-facing には、一部だけ壊れていても十分に障害です。逆に、部分障害だからこそ degrade で耐えられることもあります。`Search Indexer` が 10 分遅れても注文受付は止めない、といった判断です。

cascading failure を防ぐ原則

同期 path の障害が広がる典型パターンは次の通りです。

1. downstream が遅くなる
2. caller の queue と in-flight request が増える
3. timeout と retry が増える
4. 負荷がさらに上がる
5. upstream も巻き添えになる

これを止める原則はすでに出てきました。

- timeout budget を持つ
- retry budget を制限する
- rate / concurrency limit を置く
- circuit breaker で早めに諦める
- critical path を短く保つ

playbook として大切なのは、「障害時ほど仕事を減らす」ことです。全部守ろうとすると全部落ちます。

インシデント 1: payment timeout が急増した

checkout API の p99 latency が急増し、`Payment Service` の timeout が多発しているとします。このとき確認すべき順番は次です。

1. `Payment Service` の成功率、tail latency、queue saturation を見る
2. `Order Service` 側の retry 数と client-side queue を見る
3. 直近 deploy や config 変更の有無を確認する
4. circuit breaker が開いているか、degraded response があるかを見る
5. idempotency key により重複オーソリが増えていないか確認する

ここでのポイントは、「timeout が起きている = 支払い未実行」と決めつけないことです。response loss や遅延成功の可能性があるので、二重課金を避ける観点を必ず持ちます。

インシデント 2: analytics lag だけが増えている

注文受付は正常だが、dashboard 反映が 30 分遅れている状況を考えます。このときは Kafka 周辺に視点を移します。

1. `analytics-group` の lag を partition ごとに見る
2. 特定 partition だけ高いなら hot key や poison message を疑う
3. 全体で高いなら consumer throughput、downstream DB、batch size を疑う
4. rebalance 頻度が高いなら instance 安定性と autoscaling を疑う
5. DLQ や retry queue に偏りがないかを見る

ここで重要なのは、「Kafka が遅い」とひとまとめにしないことです。問題は broker ではなく、consumer code、partition key、外部依存にあることが多いからです。

インシデント 3: schema 変更で一部 consumer が落ちた

event schema 進化の失敗も典型です。`OrderCreated` に新 field を追加したところ、`Search Indexer` は平気だが `Email Worker` が decode error で落ちた、といった状況です。

こうした事故を防ぐには、schema governance が必要です。

- producer は後方互換性を守る
- consumer は未知 field を無視できるようにする
- 破壊的変更は version を分ける
- schema 変更をレビュー対象にする

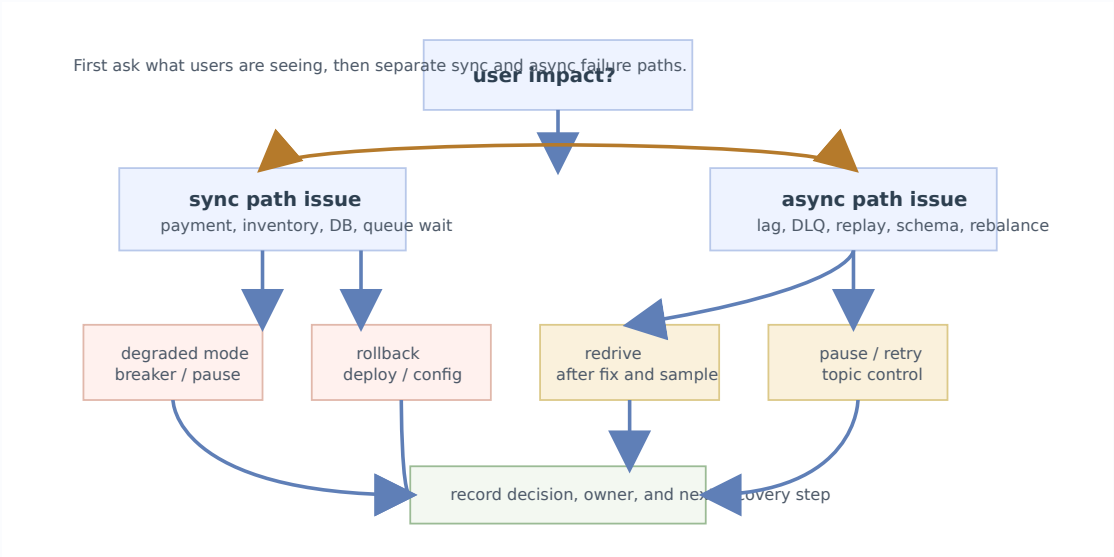
ここでも ownership が重要です。topic を誰の API と見なすかが曖昧だと、schema は簡単に壊れます。

最初の 15 分でやること

大きな障害では、最初の 15 分で 調べる順番 が重要です。継続例なら、次の順で見ると混乱しにくいです。

1. user-facing impact を確認する
2. 同期 path か非同期 path かを切り分ける
3. 直近 deploy / config change を確認する
4. retry storm や lag amplification が起きていないかを見る
5. degraded mode を使うか、完全停止するかを決める

この順序を持たずに個別 metric をつまみ食いすると、原因と影響範囲を取り違えやすくなります。



最初の 15〜30 分では、まず user impact を見てから sync path と async path を分けて止血手段を選ぶ

alert を症状と原因で分ける

運用でありがちな失敗は、alert が多すぎるのに役に立たないことです。とくに `CPU 80%`、`error log count`、`lag` のような signal を無差別に鳴らすと、症状と原因が混ざってオンコールが動きにくくなります。

整理しやすいのは、alert を次の 2 層に分けることです。

- 症状 alert user-facing impact や business SLO の悪化を検知する
- 原因候補 alert payment p99、consumer lag、DLQ 増加、schema decode error を検知する

前者は 起こっている障害 を伝え、後者は どこを疑うか を伝えます。この 2 つを分けておくと、深夜のインシデントでも優先順位を付けやすくなります。

RPC と Kafka の使い分け表

ここまでの議論を、設計時の判断表に落とすと次のようになります。

- いま結果が必要 RPC
- 複数下流へ事実を配りたい Kafka

- user-facing latency を短くしたい 必須だけ RPC、副作用は Kafka
- 強い request/response 因果が必要 RPC
- replay / backfill を前提にしたい Kafka
- ordering scope を entity 単位で持ちたい Kafka + key 設計、または idempotent RPC
- 強い一括トランザクションが欲しい service 境界を見直す。安易にどちらか一方へ期待しない

この表は万能ではありませんが、少なくとも Kafka は新しいから良い、同期のほうが単純だから良いといった雑な議論を避けやすくなります。

観測を hop ごとと end-to-end で分ける

observability では、hop ごとの健全性と end-to-end の業務体験を分けて見る必要があります。

- hop ごと RPC error rate、p99 latency、consumer lag、DLQ count
- end-to-end 注文受付からメール送信完了までの時間 注文受付から検索反映までの時間 注文受付から analytics 集計反映までの時間

hop ごとが green でも end-to-end が壊れることはあります。たとえば各 consumer が少しずつ遅く、結果として user 通知が 20 分遅れるケースです。だから business-level signal を持つことが重要です。

```
Incident dashboard order

1. Business impact
   checkout success rate, mail sent within 5m, search freshness

2. Critical path
   payment p99, inventory error rate, DB tx latency

3. Async path
   consumer lag, DLQ, retry topic depth

4. Change surface
   deploys, schema change, config rollout
```

ownership と境界

分散システムでは、通信方式の選択はチーム境界の選択でもあります。RPC はその場の契約を強くし、Kafka は時間的疎結合を増やします。その代わり Kafka は ownership が曖昧になりやすいです。

最低でも次を決める必要があります。

- topic / API の owner は誰か
- schema 変更をレビューするのは誰か
- lag / DLQ の責任を持つのは誰か
- redrive 手順を持つのは誰か
- 重大インシデント時にどのチームが pager を持つのか

技術的に正しい設計でも ownership が曖昧だと回りません。これは分散システムの難しさの一部です。

反パターン

本書の内容を逆から見ると、避けたい反パターンも見えてきます。

- なんでも同期 RPC fan-out critical path が長くなり、障害が連鎖しやすい
- なんでも Kafka 化 user-facing state が曖昧になり、product と合わなくなる
- retry を各 hop で独立に増やす 障害時に amplification が起きる
- topic を作ったら ownership を決めない schema と lag の責任が宙に浮く
- DLQ を置いたが redrive 手順がない 失敗を隠しただけになる
- replay を考えずに event payload を最小化しすぎる 後から再計算できなくなる

反パターンは、個別技術の失敗というより、設計判断を先送りした結果として起きます。

> **編集注記**
> 良い playbook は、障害時に新しい発想を要求しません。平時から「この症状ならまずここを見る」が共有されている状態を目指します。

設計レビューの観点

新しい service 間通信を設計するとき、最低限次の問いをレビュー観点として持つと事故が減ります。

1. この call / event の owner は誰か
2. response や publish 成功は何を保証するのか
3. timeout / retry / idempotency はどこで定義されるのか
4. ordering scope は何か
5. replay / backfill は必要か
6. DLQ / redrive はどうするか
7. end-to-end でどの指標を監視するか

この 7 問に答えられないまま implementation に入ると、あとでほぼ必ず戻り工事になります。

段階的に改善するには

すでに同期 RPC fan-out が肥大化している system を、いきなり event-driven に全面刷新するのは危険です。現実的には段階的に行います。

1. critical path を可視化する
2. user-facing でなくてよい副作用を洗い出す
3. outbox を置き、まずは 1 つの consumer へ流す
4. consumer を idempotent にし、lag/DLQ を観測する
5. 徐々に fan-out を同期から非同期へ移す

この順序を取ると、設計変更の効果と新しい障害面を少しずつ学べます。

runbook に残すべきこと

playbook を実運用へ落とすなら、runbook として明文化すべきです。少なくとも次は残しておく価値があります。

- 主要 API と topic の owner
- 主要 SLO と alert threshold
- DLQ / redrive 手順
- replay 実行時の注意点
- degraded mode の条件
- external provider 障害時の暫定運用

分散システムでは、知識が人に閉じると障害時に復旧速度が大きく落ちます。

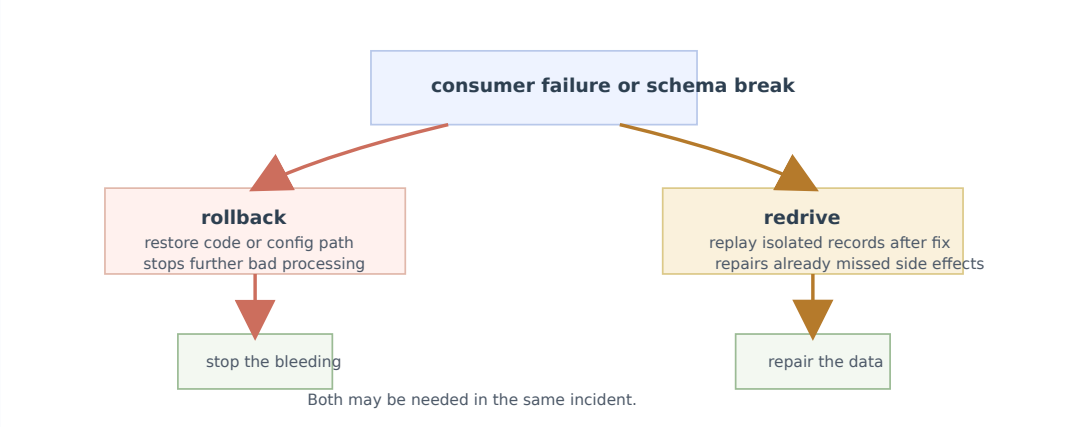
redrive と rollback を混同しない

Kafka や outbox を使い始めると、障害対応で とりあえず巻き戻す という言葉が出がちです。しかし実際には、rollback と redrive は別物です。

- rollback 変更した code や config を以前の状態へ戻す
- redrive 既に保存されている record をもう一度処理させる

schema 破壊で consumer が落ちた場合、まず必要なのは rollback かもしれません。しかし欠損したメール送信や検索更新を埋めるには、そのあとに redrive が必要です。この 2 つを混ぜると、`binary` は戻したが欠損は埋まっていない 状態で安心してしまいます。

したがって playbook には、止血 と データ修復 を分けて書くべきです。分散障害は、process を戻すだけでは終わらないことが多いからです。



rollback はこれ以上の悪化を止め、redrive は既に欠けた side effect や projection を修復する

ケーススタディ: 同期 fan-out から event 化へ

注文作成 API がもともと `Email Service` と `Search Service` を同期 `RPC` していたとします。これを Kafka へ移すときの狙いは、単に速くすることではありません。

- checkout latency を短くする
- email / search の障害を注文受付から切り離す
- replay や backfill を可能にする

ただし移行では、新たに lag、DLQ、schema governance が必要になります。つまり「何が楽になり、何が新しく難しくなるか」を理解したうえで進める必要があります。

移行のロールアウト順序

同期 fan-out から event 化へ移すときは、順序が大切です。安全なのは次の順です。

1. まず outbox だけ入れる
2. 既存同期処理は残したまま、consumer で shadow 処理する
3. lag / DLQ / replay を観測できるようにする
4. 問題ないと確認してから同期 call を外す

この順序なら、いきなり user-facing behavior を変えずに、新しい非同期経路の安定性を確認できます。

degraded mode の棚卸し

設計レビューで有効なのは、「何が落ちたらどこまで degraded で耐えるか」を一覧にすることです。たとえば継続例では次のように整理できます。

- `Inventory Service` 障害 注文受付そのものが止まり得る
- `Payment Service` 障害 注文受付そのものが止まり得る
- `Email Worker` 障害 注文受付は継続し、通知だけ遅延させる
- `Analytics Consumer` 障害 注文受付は継続し、集計だけ遅延させる
- `Search Indexer` 障害 注文受付は継続し、検索反映だけ遅延させる

この一覧があるだけで、どこを同期に残し、どこを非同期へ逃がすべきかがかなり明確になります。

architecture review worksheet

最後に、設計レビューでそのまま使える簡単な worksheet を置いておきます。

```
Communication name:
Caller:
Callee / Topic owner:

1. Success means:
2. Critical path or asynchronous side effect:
3. Timeout / deadline:
4. Retry policy:
5. Idempotency key / Event ID:
6. Ordering scope:
7. Replay / redrive requirement:
8. Owner during incidents:
9. End-to-end metric:
10. Degraded mode:
```

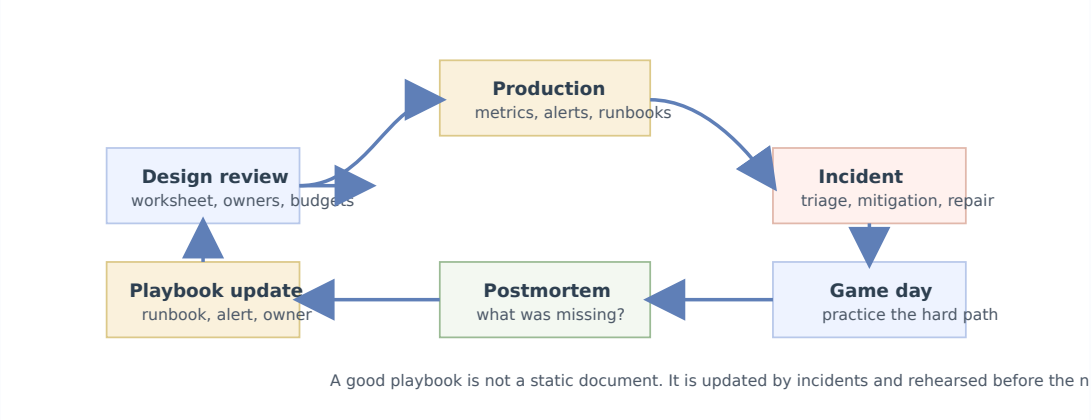
短いですが、この 10 項目に答えられれば大半の設計議論は具体的になります。

playbook をチームで運用する

playbook は文書として置くだけでは効きません。次の運用があると初めて意味が出ます。

- 新しい API / topic を追加するときに worksheet を埋める
- schema 変更レビューで ownership と replay 影響を確認する
- インシデント後に `playbook に足りなかった判断` を追記する
- オンコール訓練でケーススタディを使う

つまり playbook は静的な文書ではなく、system の学習記録です。



設計レビュー、運用、障害、postmortem、game day を回して初めて playbook は強くなる

障害演習の観点

机上の訓練でも、次の 3 題を回すだけでかなり効きます。

- payment timeout が 10 倍に増えた
- analytics lag が 1 時間に達した
- `OrderCreated` schema 変更で consumer が decode error を出した

それぞれについて、最初に見るダッシュボード、degraded mode、owner、rollback 条件を話せるかを見ると、playbook の弱点が出ます。

postmortem で残すべきもの

障害が収束したあとに重要なのは、単に時系列を残すことではありません。次に同じ障害が来たときに判断を短くする材料を残すことです。

最低限ほしいのは次です。

- 最初に見えた症状と真因のズレ
- どの alert が早すぎたか、遅すぎたか
- どの dashboard / trace が効いたか
- rollback と redrive のどちらが必要だったか
- playbook に欠けていた判断分岐

良い postmortem は責任追及の文書ではなく、次回の判断コストを下げる文書です。

30 分インシデントタイムライン

オンコールで useful なのは、抽象論より `最初の 30 分に何をするか` が決まっていることです。継続例なら、次の流れにしておくと混乱が減ります。

```
0-5 min
  user impact 確認
  checkout / mail / search / analytics のどこが壊れたか切り分ける

5-10 min
  直近 deploy と config change を確認
  retry storm, lag amplification, DLQ 増加を見る

10-20 min
```

```
  degraded mode / pause / rollback / breaker のどれで止血するか決める
  owner と連絡経路を固める

20-30 min
  redrive や provider status check など修復の入口を決める
  incident note に判断根拠を残す
```

この粒度まで落とすと、playbook が 読み物 ではなく 初動手順 になります。

on-call の役割分担を決める

分散障害では、全員が同じダッシュボードを見ているだけだと進みません。最低限、次の役割があると復旧が速くなります。

- incident commander 優先順位と次の行動を決める
- domain owner payment、Kafka、search など個別領域を深掘る
- communicator product、support、他チームへ状況を共有する

小規模チームでは 1 人が兼任してもよいですが、役割名があるだけで 誰が判断を持つか が明確になります。分散障害は技術問題であると同時に coordination 問題でもあります。

consumer lag 用の runbook 例

consumer lag は頻度が高いので、例として runbook の最小形を置いておきます。

```
Symptom:
  analytics lag > 15 min

Check:
  1. affected group / partition
  2. offset progress or stuck
  3. rebalance count
  4. downstream DB / warehouse errors
  5. DLQ / retry topic depth

Immediate action:
  - hot partition なら scale out ではなく key / batch / handler を確認
  - provider or DB failure なら retry topic or pause を検討
  - poison なら DLQ 隔離と fix を優先

Recovery:
  - sample redrive
  - replay rate limit
  - freshness metric normalisation
```

runbook は長文である必要はありません。症状、確認順、止血、回復の 4 段だけでも十分効きます。

schema rollout 前の確認

schema 事故は rollback と redrive を同時に要求しやすいので、事前確認の価値が高いです。最低限次を確認すると事故率が下がります。

1. 新 field は optional か
2. 既存 consumer が未知 field を無視できるか
3. replay 対象の旧 record でも意味が壊れないか
4. rollback 後に redrive が必要か
5. topic owner と consumer owner が両方レビューしたか

schema 変更は code diff が小さく見えても、運用影響は大きいことがあります。

game day で試す題材

playbook を強くするなら、机上レビューだけでなく小さな演習が要ります。継続例なら次の題材が扱いやすいです。

- payment provider を 10 分だけ timeout させる
- analytics-group の 1 partition を poison message で止める
- `OrderCreated` の新 field を旧 consumer に流す
- Search Indexer の replay と live traffic を同時に走らせる

目的は本番に近い pain を安全に再現し、何を見るか と どこで止血するか を練習することです。

playbook の成熟度を測る

最後に、playbook 自体の品質を測る問いを置いておきます。

1. 代表的な 3 障害について、最初の 15 分の行動が文章なしで言えるか
2. rollback と redrive の判断が runbook に分かれているか
3. owner と pager が topic / API ごとに明示されているか
4. postmortem で増えた判断が playbook へ還元されているか
5. 新メンバーでも worksheet を使って設計レビューに参加できるか

この 5 問に yes が増えるほど、playbook は文書ではなく運用資産になります。

章末まとめ

この章で大切なのは、障害対応を即興にしないことです。何を先に見るか、どこまで degraded で耐えるか、誰が owner か を先に決めておくと、技術の違いに引きずられにくくなります。

設計が良いとは、平時にきれいに見えることではなく、障害時に判断順序が共有されていることです。

この章の設計判断の要点

良い設計とは、常に成功する system を作ることではありません。失敗したときに、どこが壊れ、誰が責任を持ち、どこまで degraded で耐えられるかを先に言語化している system を作ることです。

注文システムの継続例でも、payment timeout、analytics lag、schema 破壊は普通に起きます。重要なのは、それぞれに対して 同期 path を守るのか、eventual に追いつけばよいのか、人手介入が必要なのか を決めておくことです。

> **注意**
> 分散システムでは、障害時の即興は高くつきます。設計文書の価値は、平時ではなく障害時に試されます。

演習

1. あなたの system にある通信のうち、なんでも Kafka 化 と なんでも同期 RPC のどちらに寄りすぎているかを診断してください。
2. 最も現実的なインシデントを 1 つ選び、最初の 15 分で見る指標と判断を playbook として書いてください。

3. ある topic の owner が曖昧なときに、どのような事故が起こりやすいか列挙してください。
4. end-to-end で監視すべき business metric を 3 つ定義してください。
5. あなたの system で degraded mode を持つべき機能を 3 つ挙げ、完全停止との境界を説明してください。
6. 直近のインシデントを 1 つ選び、worksheet の 10 項目で振り返ってください。

この章の出口

本編で組み立てた `同期通信と非同期通信の使い分け`、`失敗時の設計`、`ownership` と `observability` を、最後は手元で使える形へ落とします。付録では復習、用語、設計レビュー用の問い、次に読む資料をまとめます。

付録: 復習、用語、次の一歩

本編では、`RPC` と Kafka を技術名としてではなく、サービス間通信を設計するための判断軸として見てきました。この章では、復習、自己点検用の問い、用語集、設計レビュー観点、次に読む資料をまとめます。新しい topic を増やすというより、本書を読んだあとに手元の system へ戻すための章です。

この章の到達点

この章では次を押さえます。

- 各章の復習ポイント
- 自己点検用の問い
- よく使う用語の整理
- 設計レビューで使える簡易チェックリスト
- 次に深める方向

章ごとの復習ポイント

- 導入 local call の直感は分散通信では壊れる。技術名ではなく、いつ結果が必要か、失敗時に誰が責任を持つかを軸に考える。
- request/response critical path、response の意味、timeout budget、idempotency を最初に定義する。
- `RPC` の運用設計 service discovery、load balancing、queue、retry budget、breaker、tracing は同期通信の一部である。
- Kafka と log event を command と混ぜず、`topic`、`partition`、`offset`、key 設計で ordering scope を表現する。
- consumer lag、rebalancing、DLQ、redrive、idempotency が本体であり、publish 成功だけでは system は完成しない。
- semantics と consistency at-least-once / exactly-once は hop 単体でなく end-to-end で読む。outbox と saga は境界整理の道具である。
- playbook ownership、schema governance、observability、degraded mode を先に決めることが、障害時の即興を減らす。

自己点検用の問い

本書を読み終えたら、手元の system に対して次の問いに答えてみると理解が固まります。

1. いま最も user-facing な API の critical path は何か

2. その API が成功を返した時点で、何を保証しているか
3. timeout と retry はどこで定義されているか
4. idempotency key が必要な call はどれか
5. event の ordering scope は何か
6. lag が増えたとき、誰が見るべきか
7. DLQ に入った record をどう redrive するか
8. end-to-end の business metric は何か

この 8 問に答えられない箇所は、設計がまだ暗黙のまま残っている可能性が高いです。

設計レビュー用の簡易チェックリスト

新しい service 間通信を提案するとき、次のチェックリストをそのまま使えます。

- この通信は `RPC` か event か。なぜその形なのか。
- caller はいつ結果を必要としているのか。
- response / append 成功は何を保証するのか。
- timeout / deadline / retry policy はどこにあるか。
- idempotency key や `event_id` は何か。
- ordering scope は何か。
- consumer lag や DLQ の owner は誰か。
- replay / backfill が必要になったときに対応できるか。
- schema 変更の互換性ルールは何か。
- 障害時の degraded mode はあるか。

短い checklist ですが、多くの設計事故はここで検知できます。

クイック比較表

本書の後半では比較表が何度も出てきます。読み返しやすさのために、もっとも重要な比較をここへまとめておきます。

```
RPC vs Kafka

RPC
  向いている: いま結果が必要な仕事
  主な難しさ: timeout, retry, queue, cascading failure
  成功の意味: response の定義で決まる

Kafka
  向いている: response 後へ押し出したい事実の配布
  主な難しさ: ordering, lag, replay, consumer ownership
  成功の意味: append, consumer processing, user-visible effect を分けて考える
```

```
Sync path vs Async path

Sync path
  主眼: accepted / rejected / temporary_failure (= unknown to caller) を早く狭める
  代表要素: timeout budget, idempotency key, failure mapping

Async path
  主眼: side effect を疎結合に進める
  代表要素: event schema, consumer lag, DLQ, redrive
```

```
Retry family
```

```
retry
  同じ call / record を再試行する

replay
  過去 log を読み直す

redrive
  隔離済み record を修復後に再投入する

rollback
  code / config を以前の版へ戻す
```

この 3 表だけでも、いま議論しているのはどの層の問題か をかなり切り分けやすくなります。

本書での表記ルール

本文では似た名前が複数出るので、ここで表記を固定します。

- `SubmitOrder` user からの同期 request 名
- `OrderAccepted` 同期 path が成立したときの意味
- `OrderCreated` domain event の概念名
- `order-created` topic 名や log 上の event type 名
- `idempotency key` 概念としての一般名
- `IdempotencyKey` コード中の field 名
- `event_id` storage や payload の一般的な field 名
- `EventID` コード中の field 名

このルールは厳密な文法というより、概念名、`log` 上の名前、コード識別子 を混ぜないための目印です。分散システムでは、名前のズレがそのまま責務のズレになりやすいので、初学段階ほど明示したほうが読みやすくなります。

用語小事典

- critical path user-facing な request が成功と見なされるまでに必要な処理列
- timeout 一定時間待って結果が来なければ諦める規則
- deadline request 全体が完了すべき絶対期限
- retry budget 障害時に追加試行をどこまで許すかの上限
- idempotency 同じ操作を何度繰り返しても、1 回実行した場合と最終状態が同じになる性質
- service discovery 論理名から動的な instance 集合を見つける仕組み
- load balancing request を複数 instance に分配する仕組み
- circuit breaker 失敗率が高い依存先への呼び出しを一時的に止める仕組み
- event すでに起きた事実を記録するもの
- command 特定の相手にやってほしい仕事を依頼するもの
- topic 同種の record を置く Kafka の論理名
- partition 並列性と順序保証の単位
- offset partition 内の位置を表す連番
- consumer group 同じ論理 consumer を複数 instance で分担する仕組み
- lag consumer がまだ追いついていない量
- rebalance consumer group 内で partition 所有者を再配分すること
- poison message 繰り返し処理しても失敗する record
- DLQ 一定回数の再試行後も処理できなかった record を隔離する場所
- redrive DLQ や保留領域から record を再投入すること

- outbox state 変更と publish 要求を同じ transaction で記録する pattern
- saga 複数 service のローカル処理と補償で全体整合を取る発想
- replay 過去 record を再読して処理をやり直すこと
- backfill 過去欠損分をあとから埋める処理
- ordering scope 順序を守りたい単位
- end-to-end consistency 複数 hop をまたいだ業務操作全体の整合性

consumer タイプの比較

継続例で使った 3 つの consumer を、実務判断向けに簡単に並べ直しておきます。

```
Email Worker
  許容 lag: 小さめ
  replay safety: 低い
  重要事項: duplicate suppression, provider quota, notification UX

Analytics Consumer
  許容 lag: 中〜大
  replay safety: 高い
  重要事項: event time, throughput, backfill, warehouse load

Search Indexer
  許容 lag: 中
  replay safety: 比較的高い
  重要事項: upsert, rebuild, live traffic と replay の分離
```

この比較を頭に置くと、`同じ topic を読んでいるから同じ運用でよい` という誤解を避けやすくなります。

よくある誤解

`RPC` は関数呼び出しの延長にすぎない

違います。queue、connection pool、timeout、retry、partial failure を含む別物です。

Kafka は queue の高機能版である

一面ではそう使えますが、本質は複数 consumer が独立進捗で読む append-only log です。

exactly-once を使えば重複はもう考えなくてよい

成立範囲を限定しなければ危険です。外部 side effect は依然として application 設計が必要です。

DLQ に送れば安全である

安全ではありません。隔離しただけです。redrive 手順、owner、再実行時 idempotency が必要です。

lag は worker を増やせば解決する

hot partition、外部依存、poison message、rebalancing が原因なら、単純なスケールアウトでは解けません。

> **注意**
> 分散システムで危険なのは、間違った答えそのものより、暗黙の前提に気付かないことです。よくある誤解を定期的に見直すだけでも事故は減ります。

ミニケーススタディ

ケース 1: 注文確認メールが二重送信された

考えるべき問い:

- どこで retry が起きたか
- メール送信 API に idempotency key はあるか
- 送信履歴は `event_id` と結びついているか
- commit と side effect の順序はどうなっているか

ケース 2: 検索結果への反映が遅い

考えるべき問い:

- lag は全 partition で高いか、一部だけか
- index update は upsert か append か
- bulk 処理と refresh interval は妥当か
- event payload だけで処理できるか、それとも追加 `RPC` があるか

ケース 3: 注文受付は成功するが analytics が欠損する

考えるべき問い:

- outbox は書けているか
- relay は動いているか
- consumer は commit を先行していないか
- 欠損検知の metric はあるか

こうしたケースは、本書の各章を実務へ戻す入口として使えます。

設計レビュー演習

以下の 3 題は、チームで設計レビューするときの題材として使えます。

1. `OrderCreated` event から email と search を非同期化するとき、何を同期 path に残すか。
2. payment timeout 時に二重課金を避けながら user へ何を返すか。
3. analytics lag が 1 時間まで伸びても注文受付を止めないなら、どの指標を alert にするか。

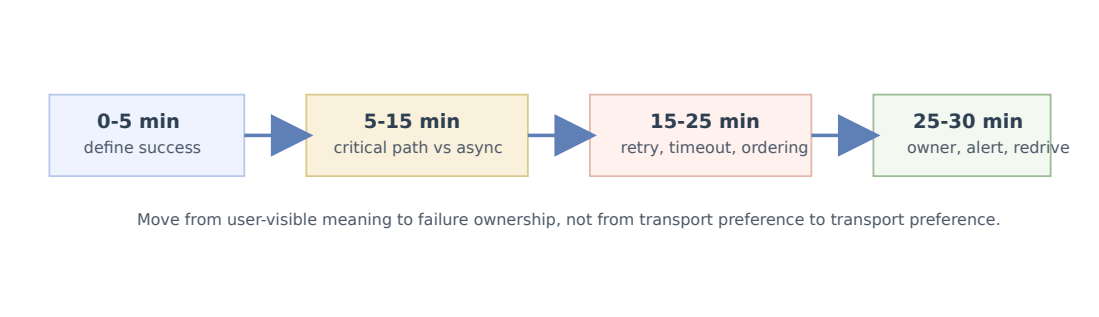
短い題材ですが、本書の中心線である `critical path`、`idempotency`、`lag`、`ownership` を一度に話せます。

30 分レビューの進め方

設計レビューは、資料を読むだけだと抽象論で終わりがちです。30 分だけ使うなら、次の順が効率的です。

1. 最初の 5 分で 成功とは何か を定義する
2. 次の 10 分で critical path と非同期 side effect を分ける
3. 次の 10 分で timeout / retry / idempotency / ordering scope を詰める
4. 最後の 5 分で owner、alert、redrive を確認する

この順なら、transport の好みより どこに失敗を押し出すか の議論へ自然に入れます。



30 分レビューでは、成功の意味から始めて、critical path、failure contract、ownership へ順に落とす

20 の確認質問

最後に、実務へ戻るときの短い確認質問を並べます。答えられない項目は、そのまま設計の曖昧さです。

1. この API の成功は何を意味するか。
2. timeout は誰が決めるか。
3. retry は誰が行うか。
4. 自動 retry してはいけない call はどれか。
5. idempotency key はどこで生成するか。
6. `event_id` は stable か。
7. ordering scope は何か。
8. replay したくなる consumer はどれか。
9. replay してはいけない consumer はどれか。
10. DLQ の owner は誰か。
11. redrive 手順はどこに書いてあるか。
12. lag の alert threshold は何に基づくか。
13. business error と technical error は分かれているか。
14. schema 変更のレビュー担当は誰か。
15. end-to-end metric は何か。
16. degraded mode は product と共有されているか。
17. topic の名前は事実を表しているか。
18. command と event を混ぜていないか。
19. critical path に本当に必要な依存先だけが入っているか。
20. 障害時に最初の 15 分で見るダッシュボードは決まっているか。

手元の system でやる小さなワーク

本書を閉じたあと、1 時間だけ使えるなら次の 4 つをやると効果があります。

1. もっとも重要な API を 1 本選び、critical path を箱書きする
2. その API の成功が何を保証するかを 3 行で書く
3. 関連する event を 1 つ選び、owner、ordering scope、redrive 手順を書き出す
4. `いま playbook が無い` と思うインシデントを 1 つ選び、最初の 15 分の行動を書いてみる

理解は読むだけでは定着しません。自分の system に引きつけて `曖昧な前提` を見つけたところから価値が出ます。

ミニケーススタディを増やす

短いケースをもう 3 つ追加します。どれも正解を暗記するためではなく、判断軸を使う練習のためのものです。

ケース 4: DLQ が増えているが user 影響はまだ見えない

考えるべき問い:

- どの consumer group の DLQ か
- 将来的に user-facing へ波及する経路はあるか
- redrive 前に code fix が必要か
- DLQ に隔離した record の寿命はどうなっているか

ケース 5: payment provider が断続的に 502 を返す

考えるべき問い:

- technical error と business reject は分かれているか
- idempotency key により再試行安全性を担保できるか
- caller と provider の timeout のどちらが先に切れているか
- degraded mode と user 向け表示をどうするか

ケース 6: replay 中だけ analytics warehouse が飽和する

考えるべき問い:

- replay rate を通常 traffic と別に制御できるか
- backfill を夜間に寄せるべきか
- replay 済み record をどう追跡するか
- event payload だけで処理できるか、追加読みが詰まりを作っていないか

設計の危険信号

設計レビューで `この会話が出たら一段立ち止まる` という危険信号もあります。継続例に限らず、次はかなり強い warning です。

- `とりあえず全部 retry しておく`
- `DLQ に送るので大丈夫`
- `event は ID だけにして、足りない分は後で RPC する`
- `owner は特に決めなくても使う人が見る`
- `exactly-once があるので重複は考えなくてよい`
- `lag は台数を増やせばそのうち解決する`

こうした言い方が出たら、たいてい failure contract か ownership がまだ曖昧です。

設計メモの最小テンプレート

新しい API や topic を提案するとき、長い設計書がなくても次の形だけでかなり議論できます。

```
Title:
Owner:

User-visible success:
Critical path:
Async side effects:
Failure modes:
Retry / idempotency:
```

```
Ordering scope:
Replay / redrive:
Primary metrics:
```

これを 1 ページで書けない場合、実装前にまだ決めるべきことが残っている可能性が高いです。

runbook の最小テンプレート

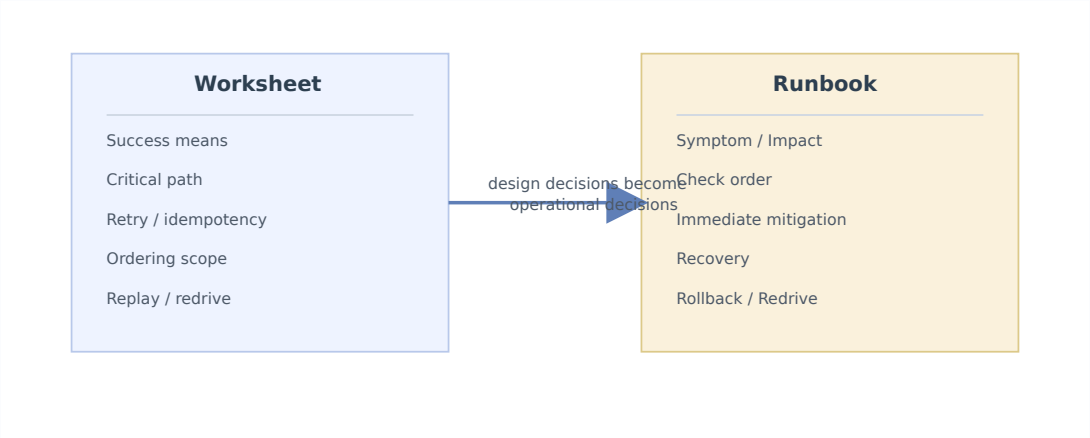
runbook も同じで、最初から大きくしすぎるより最小形を持つほうが効きます。

```
Symptom:
Impact:
Owner:

Check order:
1.
2.
3.

Immediate mitigation:
Recovery:
Rollback needed? yes / no
Redrive needed? yes / no
```

この型を topic / API ごとに埋めていくと、障害時の即興がかなり減ります。



worksheet は設計時の意味を固定し、runbook は障害時の確認順と回復手順を固定する

90 分ワークショップ案

チームで本書の内容を実務へ結び付けるなら、90 分の短いワークショップがやりやすいです。

1. 20 分 もっとも重要な user-facing API の critical path を書く
2. 20 分 主要 event の owner、ordering scope、replay 需要を書く
3. 20 分 payment timeout か consumer lag のどちらかを題材に初動を話す
4. 20 分 worksheet と runbook を 1 つずつ埋める
5. 10 分 playbook に無い判断を洗い出す

この進め方だと、抽象論で終わらず、実際の system に戻りやすいです。

学習を深める順序

本書のあとに何を読むかは、伸ばしたい方向で変わります。

- `RPC` を深める service discovery、service mesh、hedged requests、adaptive concurrency
- Kafka を深める replication、ISR、transactions、compaction、KRaft
- consistency を深める event sourcing、CQRS、workflow engine、transactional messaging
- observability を深める distributed tracing、SLO、error budget、log correlation
- 組織運用を深める schema registry、API review、incident response、runbook 整備

最初から全方向へ行く必要はありません。実務で一番痛い箇所から掘るのがよいです。

章別の読み返し順

手元で特定の問題にぶつかったときは、最初から読み返す必要はありません。次の順が効率的です。

- timeout / retry / 二重実行の問題 `02 -> 03 -> 06`
- lag / DLQ / consumer 停滞の問題 `05 -> 07 -> 08`
- ordering / replay / outbox の問題 `04 -> 06 -> 07`
- 設計レビューや新規機能追加 `01 -> 07 -> 08`

参考資料

- `Designing Data-Intensive Applications`
- `Building Microservices`
- Kafka の公式 document
- `gRPC` の公式 document
- `Google SRE Book`
- OpenTelemetry の公式 document
- transactional outbox pattern や saga pattern の各種解説

> **編集注記**
> 技術書を読んだあとに本当に価値が出るのは、手元の system に問いを持ち帰れたときです。付録はそのためにあります。

手元で使う 1 ページ worksheet

最後に、紙 1 枚で済む最小 worksheet を置いておきます。新しい API や topic を追加するとき、この 8 項目だけでも書いておくと会話がかなり具体化します。

```
Communication:
Owner:
User-facing? yes / no

Success means:
Critical path budget:
Retry + idempotency rule:
Ordering scope:
Replay / redrive plan:
Primary alert:
```

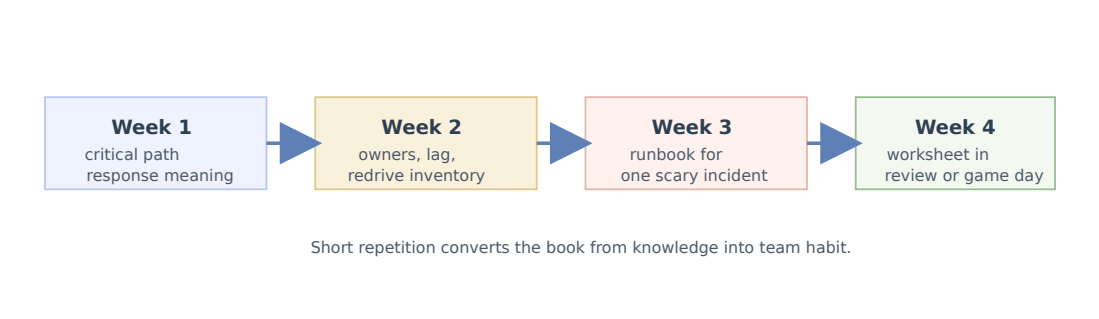
短すぎるように見えますが、短いからこそ毎回書けます。分散設計では、立派な文書より毎回更新される小さな文書のほうが効くことが多いです。

4 週間の使い方

この本を読み終えたあと、4 週間だけでも次の使い方をすると定着しやすいです。

1. 第 1 週 critical path と response meaning を手元の API で 1 本書く
2. 第 2 週 主要 topic / consumer の owner、lag、redrive を棚卸しする
3. 第 3 週 もっとも怖いインシデント 1 件の runbook を作る
4. 第 4 週 設計レビューか game day で worksheet を実際に使う

知識を 知っている から 使える に変えるには、短い反復が一番効きます。



読み終えたあとに 4 週間だけでも回すと、critical path、owner、runbook、review の習慣が残りやすい

おわりに

サービス間通信を理解するために必要なのは、個別 product の feature list を覚えることではありません。同期 request と非同期 event が、それぞれ何を保証し、どこへ難しさを押し出し、失敗時に誰へ責任を残すのかを言葉にできることです。

一度この地図を持つと、新しい broker や RPC framework を見ても、「いま解こうとしているのは timeout の問題か、ordering の問題か、consumer ownership の問題か」と整理しやすくなります。そこまで来れば、技術名は流行りのラベルではなく、目的に応じた道具として選びやすくなります。